Contents

Index By Block **1**

RAM and Arrays **2**
- Arrays in Memory . . . 2
- Binary Representation Example . . . 2
- Addresses and Element Size . . . 2
  - Storing an Integer Array in RAM . . . 2
  - Storing a Character Array in RAM . . . 3
- Key Takeaway . . . 3

Static Arrays **4**
- 1) Reading From an Array . . . 4
  - Important Clarification About $O(1)$ . . . 4
- 2) Traversing an Array . . . 5
- 3) Deleting From an Array . . . 5
  - Deleting From the End . . . 5
  - Deleting at an I-th Index . . . 7
- 4) Insertion . . . 8
  - Inserting at the End . . . 8
  - Inserting at the I-th Index . . . 8
- Static Array Time Complexity Summary . . . 9
- NeetCode Practice (Static Arrays) . . . 9
  - 1) Max Consecutive Ones . . . 9
  - 2) Remove Element (Two Pointers, In-Place) . . . 10
  - 3) Replace Elements with Greatest Element on Right Side . . . 11
  - Quick Study Tip . . . 12
  - Optional Follow-up . . . 13

Dynamic Arrays **13**
- Push to the End (Append) . . . 13
- How Resize Works . . . 13
- Amortized $O(1)$ Insertion . . . 14
- Why Doubling Capacity Works . . . 15
- Other Operations . . . 16
- Dynamic Array Time Complexity Summary . . . 16
- NeetCode Practice (Dynamic Arrays) . . . 16
  - 1) Concatenation of Array . . . 16

Stacks **17**
- Push . . . 18
- Pop . . . 18
- Peek . . . 18
- Stack Time Complexity Summary . . . 19
- NeetCode Practice (Stacks) . . . 20
  - 1) Baseball Game . . . 20
  - 2) Valid Parentheses . . . 21
  - 3) Min Stack . . . 22

Singly Linked Lists **26**

Creating a Linked List . . . 26
Traversal . . . 27
Circular Linked List . . . 27
Operations . . . 27
Appending . . . 29
Deleting . . . 29
Singly Linked List Time Complexity Summary . . . 30
NeetCode Practice (Singly Linked Lists) . . . 30
1) Reverse Linked List . . . 30

Doubly Linked Lists **35**
Insertion at the End . . . 35
Deletion at the End . . . 36
Access Pattern . . . 37
Time Complexity Summary . . . 37

Queues **37**
Implementation and Core Operations . . . 37
Enqueue . . . 38
Dequeue . . . 38
Deque Variation . . . 39
Common Use Case . . . 39
Time Complexity Summary . . . 39

Recursion (One Branch) **41**
Intuition . . . 41
Factorial Example . . . 41
Walkthrough: `factorial(5)` . . . 42
Time and Space Complexity . . . 42
Iteration vs Recursion . . . 43
NeetCode Practice (Recursion) . . . 43
1) Reverse Linked List . . . 43

Recursion (Two Branch) **44**
Fibonacci Sequence . . . 44
Walkthrough: `fibonacci(5)` . . . 46
Complexity of Naive Recursion . . . 46
NeetCode Practice (Recursion to DP) . . . 46
1) Climbing Stairs . . . 46
Approach 1: Top-Down DP (Memoization) . . . 47
Approach 2: Bottom-Up DP . . . 48
Approach 3: DP (Space Optimized) . . . 48

Insertion Sort **51**
Concept . . . 51
Stability . . . 53
Time and Space Complexity . . . 53
Time . . . 53
Space . . . 53
NeetCode Practice (Sorting) . . . 53
1) Insertion Sort (Intermediate States) . . . 53

Merge Sort **55**
Core Idea . . . 55
Merge Function (Three Pointers) . . . 56
Time and Space Complexity . . . 58
Time . . . 58
Space . . . 58
Stability . . . 58
Merge Sort vs Insertion Sort . . . 59
NeetCode Practice (Sorting) . . . 59
1) Merge Sorted Array . . . 59
Approach 1: Three Pointers With Extra Space . . . 59
Approach 2: Three Pointers Without Extra Space (Version I) . . . 60
Approach 3: Three Pointers Without Extra Space (Version II) . . . 61
Common Pitfalls . . . 61
Your Version (Same In-Place Backward Merge) . . . 62

Quick Sort **63**
Pivot Selection . . . 63
Core Idea . . . 63
Partition Step . . . 63
Time and Space Complexity . . . 65
Time . . . 65
Space . . . 65
Stability . . . 65
NeetCode Practice (Sorting) . . . 65
1) K Closest Points to Origin . . . 65
Approach 1: Sorting . . . 66

Bucket Sort **66**
Core Idea . . . 67
How the Pointers Work . . . 67
Time Complexity . . . 68
Stability . . . 68
Closing Notes . . . 69
NeetCode Practice (Sorting) . . . 69
1) Sort Colors . . . 69
Approach 1: Counting Sort . . . 69

Sorting Time Complexity Cheat Sheet **70**
Visual Summary . . . 71
How to Use It . . . 71

Binary Search (Search an Array) **73**
Core Idea . . . 73
Example Walkthrough . . . 73
Implementation . . . 73
Target Not Found . . . 75
Time and Space Complexity . . . 75
NeetCode Practice (Binary Search) . . . 77
1) Binary Search . . . 77

2) Search Range . . . 78
3) Search a 2D Matrix . . . 79

Binary Search (Search Range) 80
Concept . . . 80
Range Binary Search . . . 81
Time and Space Complexity . . . 81
NeetCode Practice (Binary Search) . . . 81
1) Guess Number Higher or Lower . . . 81
2) Search Range . . . 83
3) Koko Eating Bananas . . . 84
Approach 1: Binary Search . . . 84

Binary Trees 87
Concept . . . 87
Tree Terminology . . . 87
Root Node . . . 87
Leaf Nodes . . . 87
Height . . . 87
Depth . . . 88
Ancestor and Descendant . . . 88
Ancestor . . . 89
Descendant . . . 89

Binary Search Trees 89
Binary Trees vs Binary Search Trees . . . 89
Why BSTs . . . 89
BST Search . . . 89
How It Works . . . 90
Time Complexity . . . 91

Insertion and Removal in a Binary Search Tree 91
Insertion . . . 91
Removal . . . 92
Case 1: 0 or 1 Child . . . 92
Case 2: 2 Children . . . 93
Time and Space Complexity . . . 94
NeetCode Practice (BST) . . . 94
1) Insert into a Binary Search Tree . . . 95
2) Delete Node in a BST . . . 98

Depth-First Search 101
Concept . . . 101
DFS Traversal Orders . . . 102
Inorder Traversal . . . 102
Preorder Traversal . . . 102
Postorder Traversal . . . 102
Time and Space Complexity . . . 106
Time . . . 106
Space . . . 106
NeetCode Practice (Trees) . . . 106

1) Kth Smallest Integer in BST 106
Approach 1: Iterative DFS (Optimal) 107

DSA for Beginners

Printable Study Booklet

Generated on 2026-08-09

Index By Block

Block	Page Range
1. Arrays	2–24
2. Linked Lists	26–39
3. Recursion	41–49
4. Sorting	51–71
5. Binary Search	73–85
6. Trees	87–108

1. Arrays

RAM and Arrays

Data structures are a way to store data efficiently inside a computer component called **Random Access Memory (RAM)**.

Because RAM supports direct indexing by address, we can access any element in an array in **constant time,** $O(1)$.

RAM is often just referred to as *memory*. Let's understand how an array is stored in RAM.

Arrays in Memory

An array is an ordered collection of **contiguous** elements, for example: `[1, 3, 5]`.

However, computers represent everything as bits: `0`s and `1`s.

Most computers today have gigabytes (GBs) of RAM. For example, your machine might have **8 GB** (about 10^9 **bytes** per GB). A byte is made of exactly **8 bits**.

Binary Representation Example

Each integer in `[1, 3, 5]` has a binary representation:

- `1` -> `00000001`
- `3` -> `00000011`
- `5` -> `00000101`

If we use a single byte to store each value, those are the exact bit patterns.

When we store an array in RAM, each element is stored **continuously** and in order.

Addresses and Element Size

Integers commonly occupy **4 bytes (32 bits)** in memory.

When an integer is stored in RAM, it is associated with:

- an **address** (its location in memory), and
- a **value**.

Since array elements are contiguous, each next element is placed right after the previous one.

Storing an Integer Array in RAM

Each integer occupies 4 bytes, so addresses are **4 bytes apart**.

Instead of integers, we could also store characters in an array.

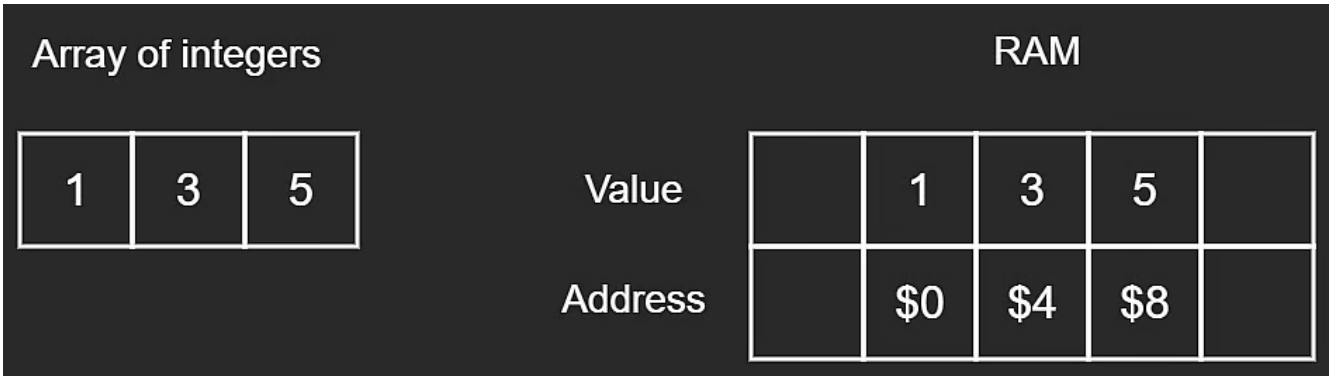


Figure 1: Integer array in RAM

Storing a Character Array in RAM

Each character takes **8 bits = 1 byte**, so addresses are **1 byte apart**.



Figure 2: Character array in RAM

Each character occupies 1 byte of space, hence addresses are 1 byte apart.

Key Takeaway

The size or type of stored data does not fundamentally change the model, as long as address increments are relative to the data type's size.

Most of this is abstracted away in day-to-day development, but this mental model is still very relevant for understanding algorithms, data structures, and *system design*.

Static Arrays

In statically typed languages like **Java**, **C++**, and **C#**, arrays must have a fixed **size** and **type** when initialized. These are called **static arrays**.

They are called *static* because their size cannot change after declaration. Once the array is full, it cannot store additional elements.

Some dynamically typed languages (such as Python and JavaScript) do not expose static arrays directly. They use alternatives, which are discussed in the next lesson.

Let's cover key array operations and the time complexity of each.

1) Reading From an Array

To read an element, we access it by index.

```
// initialize myArray
let myArray = [1, 3, 5];

// access an arbitrary element, where i is the index of the desired value
myArray[i];
```

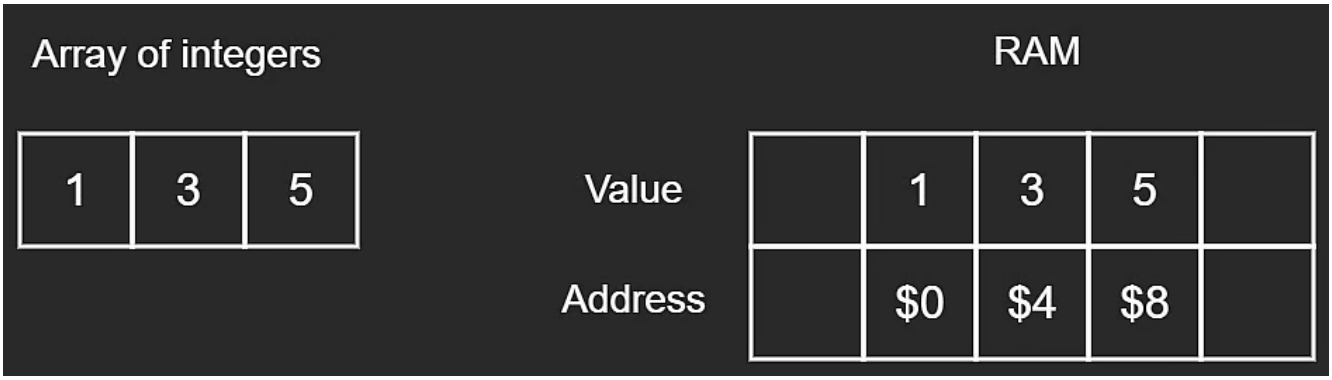


Figure 3: Accessing an element

Accessing one array element is constant time because each index maps directly to a RAM address.

- Time complexity: $O(1)$

Important Clarification About $O(1)$

There is a common confusion that $O(1)$ **always means fast**. Not necessarily.

An operation could do 1000 steps and still be $O(1)$ if that number does **not grow** with input size. Big-O measures growth, not absolute runtime.

2) Traversing an Array

To read all values, we traverse from start to end.

```
for (let i = 0; i < myArray.length; i++) {
  console.log(myArray[i]);
}

// OR

let i = 0;
while (i < myArray.length) {
  console.log(myArray[i]);
  i++;
}
```

The last element is always at index $n - 1$, where n is array size.

If size is 3, the last valid index is 2.

Traversing an array of size n takes:

- Time complexity: $O(n)$

The number of operations grows linearly with array size (10 elements -> about 10 visits, 100 elements -> about 100 visits, etc.).

3) Deleting From an Array

Deleting From the End

In statically typed languages, array slots are typically initialized with default values (like `0`, `null`, etc.).

When removing the last element, we usually perform a **soft delete**:

- overwrite with a default/sentinel value (`0`, `null`, `-1`, etc.), and
- decrement logical length by 1.

```
// Remove from the last position in the array if the array
// is not empty (i.e. length is non-zero).
removeEnd(arr, length) {
    if (length > 0) {
        // Overwrite last element with some default value.
        // We would also consider the length to be decreased by 1.
        arr[length - 1] = 0;
        length--;
    }
}
```

In `[4, 5, 6]`, value `6` is overwritten by `0` or `-1` to mark it removed, and length becomes one less.

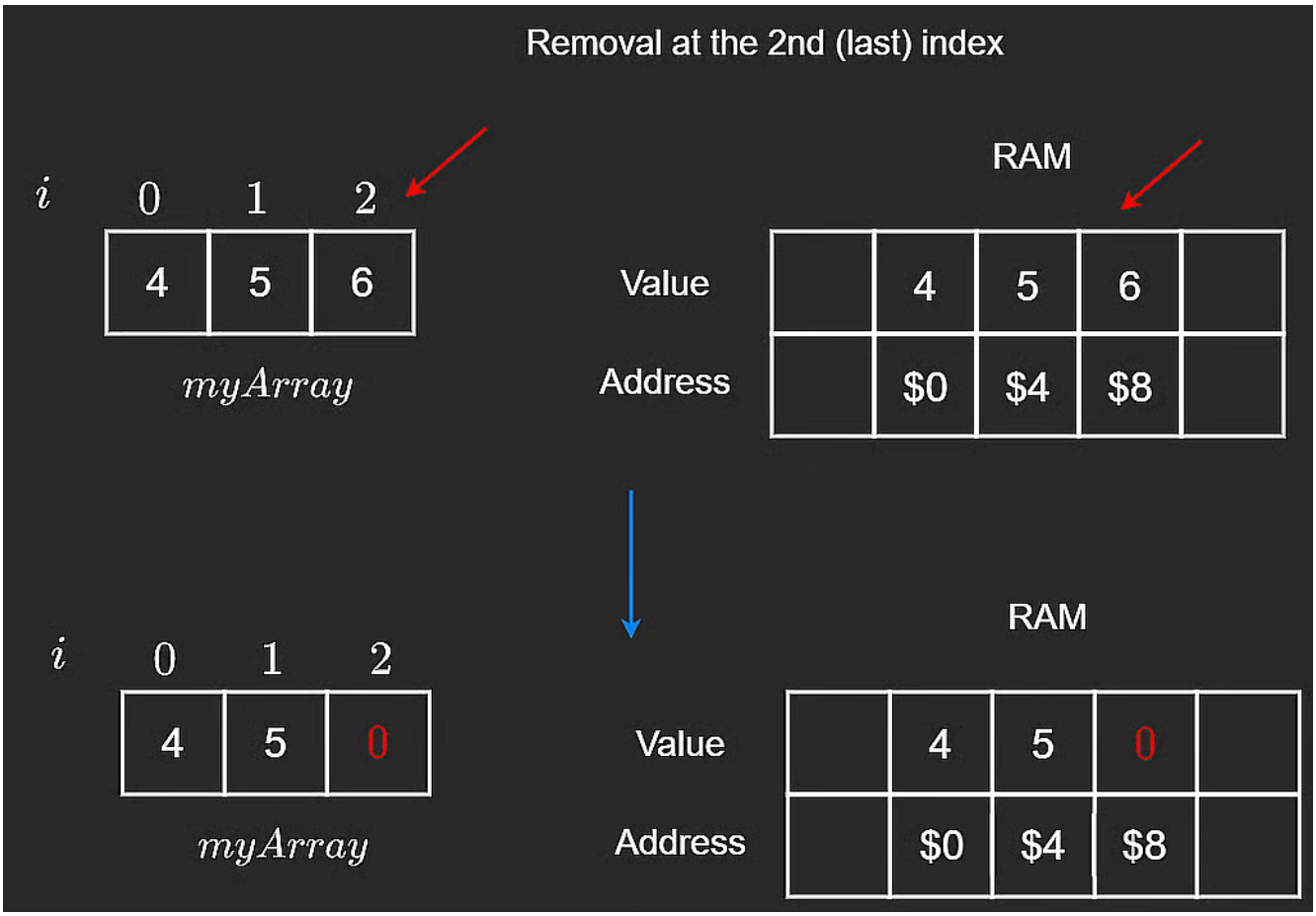


Figure 4: Removal from the end

Deleting at an I-th Index

If we delete at a random index `i`, we cannot just replace with `0`, because that would break contiguity in the middle.

Deleting from the end preserves contiguity. Deleting from the middle requires shifting.

Better approach:

1. Given deletion index `i`.
2. Iterate from `i + 1` to end.
3. Shift each element one position left.
4. Optionally set last slot to `0`/`null` and decrement length.

```
// Remove value at index i before shifting elements to the left.
// Assuming i is a valid index.
removeMiddle(arr, i, length) {
    // Shift starting from i + 1 to end.
    for (let index = i + 1; index < length; index++) {
        arr[index - 1] = arr[index];
    }
    // No need to 'remove' arr[i], since we already shifted
}
```

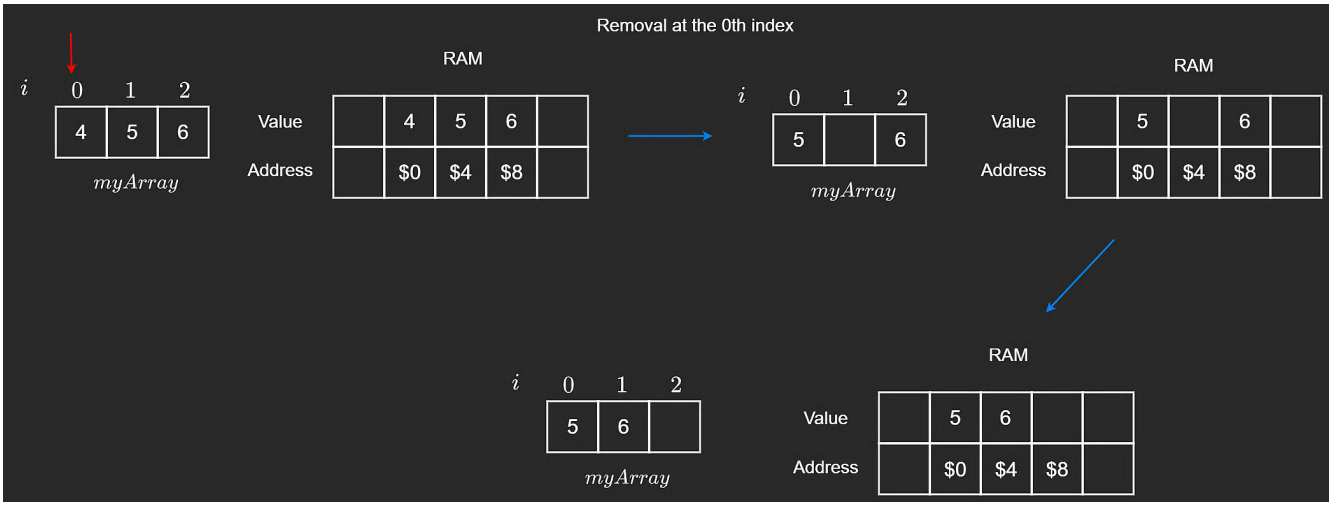


Figure 5: Removal at the 0th index

Worst case: deleting the first element requires shifting nearly all elements.

- Time complexity: $O(n)$

4) Insertion

Inserting at the End

To insert at the end, place the new value at index `length` (next free slot).

```
// Insert n into arr at the next open position.
// Length is the number of 'real' values in arr, and capacity
// is the size (aka memory allocated for the fixed size array).
insertEnd(arr, n, length, capacity) {
    if (length < capacity) {
        arr[length] = n;
    }
}
```

This writes one value:

- Time complexity: $O(1)$

Note: `length` is current element count, while `capacity` is maximum storable elements.

Inserting at the I-th Index

Inserting at arbitrary index `i` is more involved because insertion may happen in the middle.

Example: array `[4, 5, 6]`, insert at `i = 1` (or `i = 0`).

We cannot overwrite existing values, so we shift values one step right, starting from the end down to `i`.

```
// Insert n into index i after shifting elements to the right.
// Assuming i is a valid index and arr is not full.
insertMiddle(arr, i, n, length) {
    // Shift starting from the end to i.
    for (let index = length - 1; index > i - 1; index--) {
        arr[index + 1] = arr[index];
    }
    // Insert at i
    arr[i] = n;
}
```

The image below shows inserting `8` at index `1` in `[4, 5, 6]`. If there is no extra capacity, last element `6` is lost.

This demonstrates why shifting must happen **before** insertion to avoid overwriting.

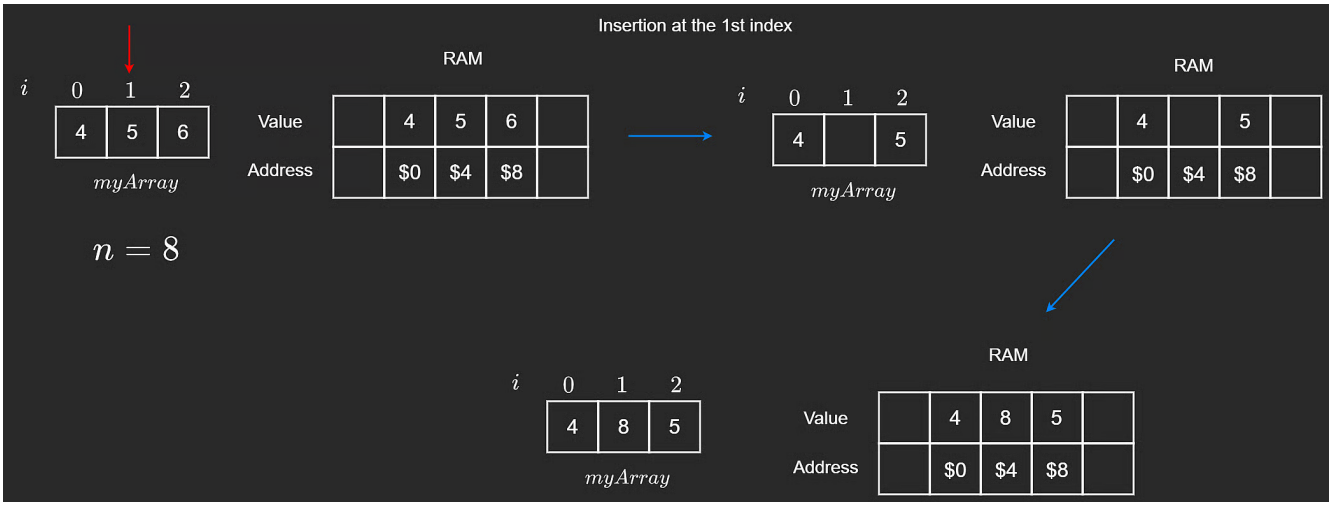
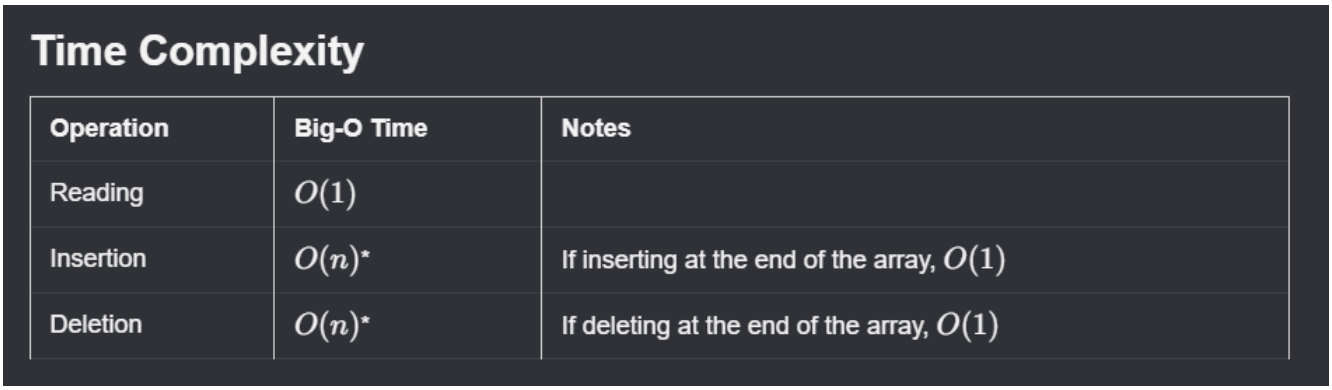


Figure 6: Insertion at the 1st index

Static Array Time Complexity Summary



Time Complexity

Operation	Big-O Time	Notes
Reading	$O(1)$	
Insertion	$O(n)^*$	If inserting at the end of the array, $O(1)$
Deletion	$O(n)^*$	If deleting at the end of the array, $O(1)$

Figure 7: Time complexity for static arrays

NeetCode Practice (Static Arrays)

Use the problems below in pattern progression order: **single pass -> two pointers (in-place) -> suffix maximum (right-to-left)**.

1) Max Consecutive Ones

- Difficulty: **Easy**
- Pattern: **Single pass + running counter**
- Core idea: Track the current streak of `1`s. Reset on `0`. Keep the best streak seen so far.

Problem statement (summary): Given a binary array `nums`, return the maximum number of consecutive `1`s.

Example:

- Input: `nums = [1,1,0,1,1,1]`
- Output: `3`

Complexity:

- Time: $O(n)$
- Space: $O(1)$

Intuition: Update the running streak on every element. If the value is `1`, increment the streak; otherwise, reset it to `0`. After each step, update the best answer seen so far.

Algorithm:

1. Initialize `res = 0` and `cnt = 0`.
2. Traverse `nums` from left to right.
3. If current value is `1`, do `cnt++`; else set `cnt = 0`.
4. Update `res = Math.max(res, cnt)`.
5. Return `res`.

TypeScript solution:

```
class Solution {
  findMaxConsecutiveOnes(nums: number[]): number {
    let res = 0;
    let cnt = 0;

    for (const num of nums) {
      cnt = num === 1 ? cnt + 1 : 0;
      res = Math.max(res, cnt);
    }

    return res;
  }
}
```

2) Remove Element (Two Pointers, In-Place)

- Difficulty: **Easy**
- Pattern: **Two pointers + in-place overwrite**
- Core idea: Keep a valid range `[0, n)`. When `nums[i] === val`, replace it with the last valid element (`nums[n - 1]`) and shrink `n`. Only advance `i` when current value is kept.

Problem statement (summary): Given an integer array `nums` and integer `val`, remove all occurrences of `val` in-place. Return `k`, the number of elements not equal to `val`.

Example:

- Input: `nums = [3,2,2,3]`, `val = 3`
- Output: `k = 2`, `nums = [2,2,_,_]`

Complexity:

- Time: $O(n)$
- Space: $O(1)$

Intuition: When a value equals `val`, replace it with the last value in the current valid range and shrink the range. This avoids unnecessary shifting and keeps the operation in-place.

Algorithm:

1. Initialize `i = 0` and `n = nums.length`.
2. While `i < n`:
3. If `nums[i] === val`, set `nums[i] = nums[n - 1]`, then decrement `n`.
4. Otherwise increment `i`.
5. Return `n` (count of kept elements).

TypeScript solution:

```
class Solution {
  removeElement(nums: number[], val: number): number {
    let i = 0;
    let n = nums.length;

    while (i < n) {
      if (nums[i] === val) {
        nums[i] = nums[n - 1];
        n--;
      } else {
        i++;
      }
    }

    return n;
  }
}
```

3) Replace Elements with Greatest Element on Right Side

- Difficulty: **Easy**
- Pattern: **Suffix maximum (right-to-left traversal)**
- Core idea: Traverse from right to left while maintaining `rightMax`. At each index, write current `rightMax`, then update it with the current value.

Problem statement (summary): Given an array `arr`, replace every element with the greatest element among elements to its right, and replace the last element with `-1`.

Example:

- Input: `arr = [2,4,5,3,1,2]`

- Output: `[5,5,3,2,2,-1]`

Complexity:

- Time: $O(n)$
- Space: $O(1)$ auxiliary space (output array not counted)

Intuition: Scan from right to left while tracking `rightMax`. For each index, the current `rightMax` is exactly the greatest value to that element's right.

Algorithm:

1. Create `ans` with same length as `arr`.
2. Set `rightMax = -1`.
3. Traverse indices from `n - 1` down to `0`.
4. Set `ans[i] = rightMax`.
5. Update `rightMax = Math.max(rightMax, arr[i])`.
6. Return `ans`.

TypeScript solution:

```
class Solution {
  replaceElements(arr: number[]): number[] {
    const n = arr.length;
    const ans = new Array<number>(n);
    let rightMax = -1;

    for (let i = n - 1; i >= 0; i--) {
      ans[i] = rightMax;
      rightMax = Math.max(rightMax, arr[i]);
    }

    return ans;
  }
}
```

Quick Study Tip

When reviewing static arrays, ask this first:

1. Can I solve it in one pass?
2. If not, can two pointers help me do it in-place?
3. If not, should I scan from right to left with a suffix value?

Optional Follow-up

If you want additional practice, rewrite each of the three solutions to:

1. Use different variable names and still explain the same invariant.
2. Add one tricky edge case test input.
3. State the invariant in one interview-ready sentence.

Dynamic Arrays

Dynamic arrays are a common alternative to static arrays because they can grow as elements are added.

In JavaScript and Python, these are the default arrays. Unlike static arrays, dynamic arrays do not require you to manually set a fixed size at initialization.

Some languages still start with a default internal capacity (for example, Java and C#), but that capacity grows automatically at runtime.

Push to the End (Append)

When appending, the new element is placed at the next free index.

```
// Insert n at the end of the array
pushBack(n) {
    if (this.length === this.capacity) {
        this.resize();
    }

    // Insert at next empty position
    this.arr[this.length] = n;
    this.length++;
}
```

If the array is full, we resize first.

How Resize Works

When capacity is reached, implementations usually **double** capacity. This reduces how often expensive resize operations happen.

Resizing creates a new, larger static array and copies existing elements into it.

```
resize() {
    // Create new array with double capacity
    this.capacity = 2 * this.capacity;
    const newArr = new Array(this.capacity);

    // Copy elements
    for (let i = 0; i < this.length; i++) {
        newArr[i] = this.arr[i];
    }

    this.arr = newArr;
}
```

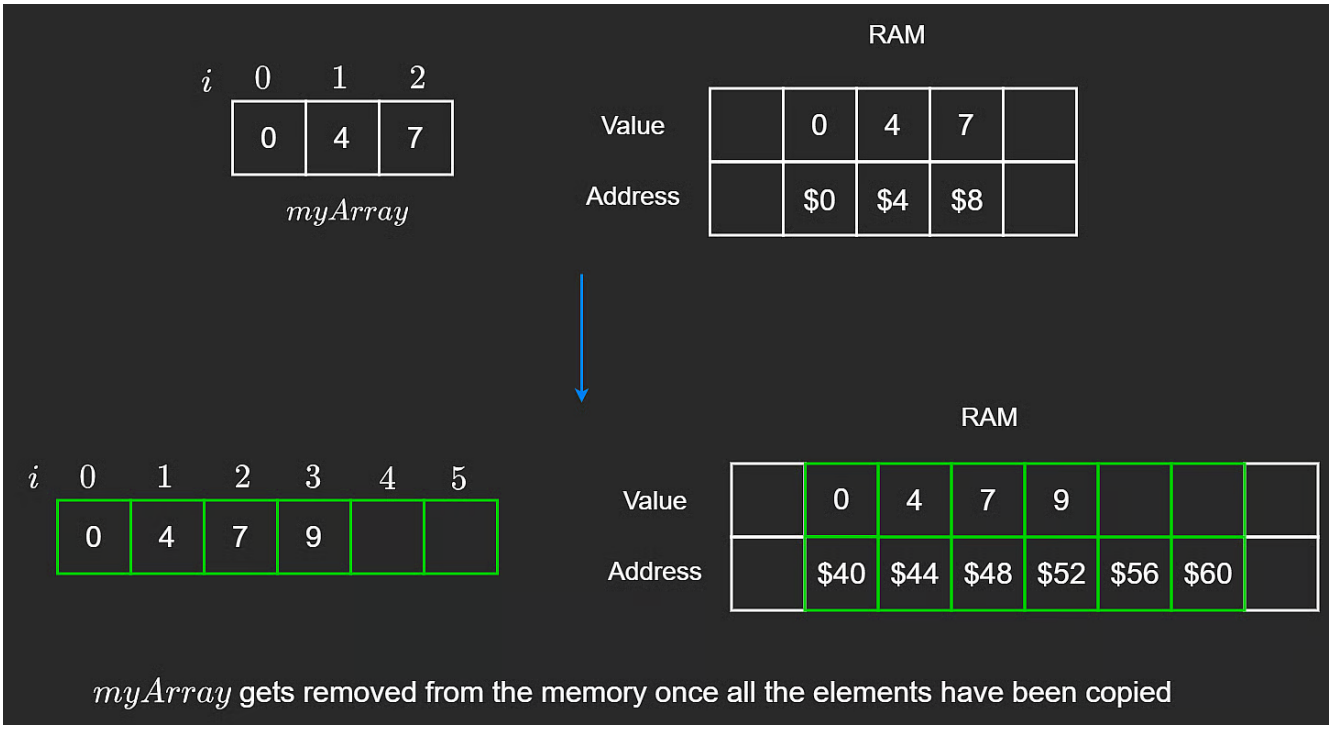


Figure 8: Dynamic array resize copy process

After copying completes, the old underlying array can be deallocated.

Amortized $O(1)$ Insertion

Appending to a dynamic array is $O(1)$ **amortized**.

Amortized time means average cost per operation over a sequence of operations.

- A resize event is expensive: $O(n)$
- Most appends are cheap: $O(1)$
- Averaged across many appends, each append is $O(1)$ **amortized**

Why Doubling Capacity Works

Suppose capacity grows as: `1 -> 2 -> 4 -> 8`.

To reach size 8, total copied/placed work follows a geometric sum:

$$1 + 2 + 4 + 8 = 15$$

And $15 < 2 \cdot 8$, so for general size n, total work is bounded by about $2n$, which is still linear:

$$O(2n) = O(n)$$

That is why inserting n elements overall is $O(n)$, and per insertion (amortized) is $O(1)$.

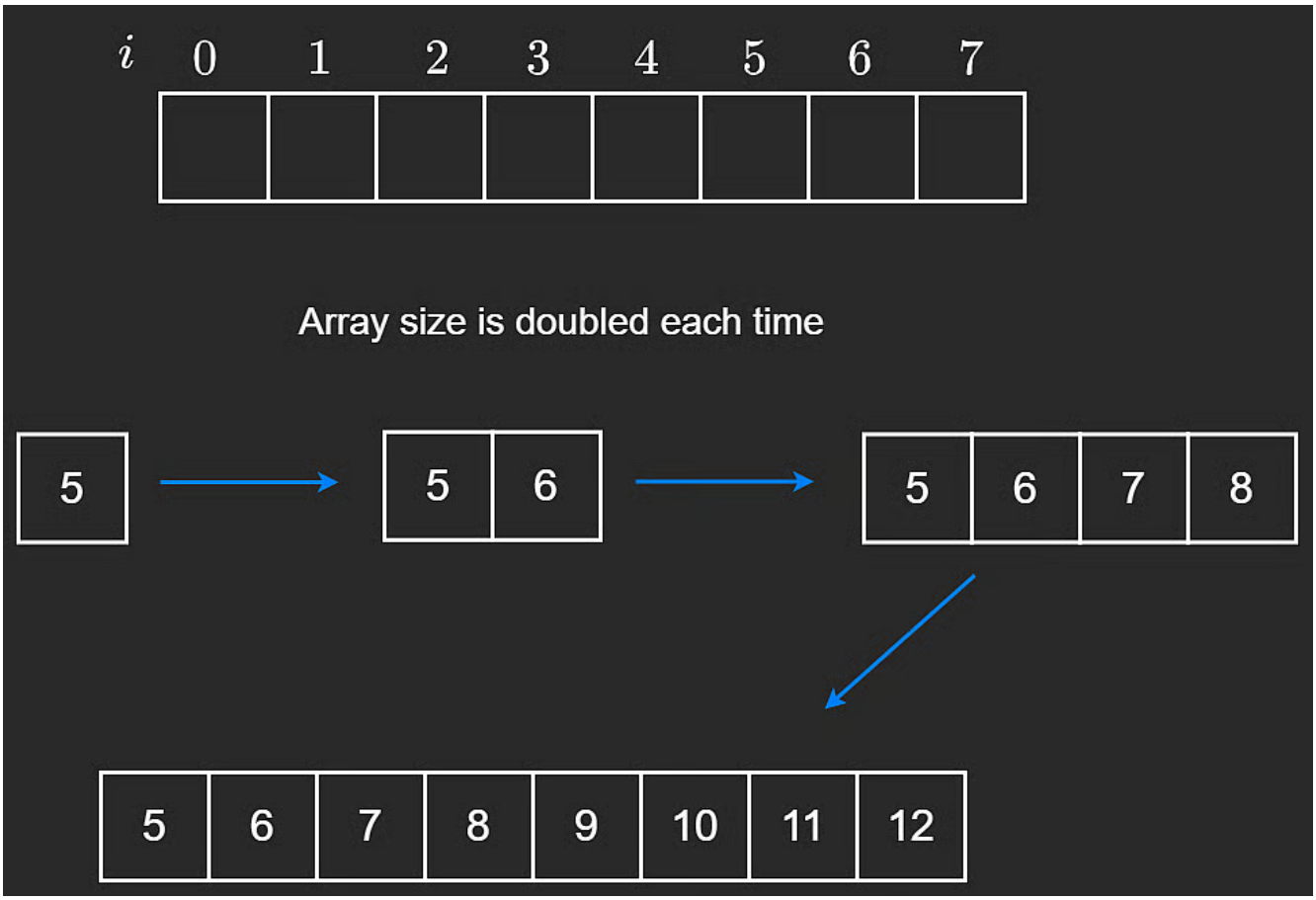


Figure 9: Dynamic capacity doubling visualization

Other Operations

Inserting or removing from the **middle** of a dynamic array is similar to static arrays: elements must be shifted left or right.

- Middle insertion: $O(n)$
- Middle deletion: $O(n)$

Dynamic Array Time Complexity Summary

Time Complexity

Operation	Big-O Time	Notes
Access	$O(1)$	
Insertion	$O(1)*$	$O(n)$ if insertion in the middle since shifting will be required
Deletion	$O(1)*$	$O(n)$ if deletion in the middle since shifting will be required

Figure 10: Time complexity for dynamic arrays

NeetCode Practice (Dynamic Arrays)

1) Concatenation of Array

- Difficulty: **Easy**
- Pattern: **One-pass index mapping**
- Core idea: Build a new array of size `2n`, and for each index `i`, write the same value to `ans[i]` and `ans[i + n]`.

Problem statement (summary): Given an integer array `nums` of length `n`, create an array `ans` of length `2n` such that:

- `ans[i] = nums[i]`
- `ans[i + n] = nums[i]`

for `0 <= i < n`.

Return `ans`.

Example:

- Input: `nums = [1,4,1,2]`
- Output: `[1,4,1,2,1,4,1,2]`

Intuition: Instead of looping twice, we can fill both required positions during the same iteration. For each `nums[i]`, write it once in the first half and once in the second half.

Algorithm:

1. Let `n = nums.length`.
2. Create `ans` with size `2 * n`.
3. Loop `i` from `0` to `n - 1`:
4. Set `ans[i] = nums[i]`.
5. Set `ans[i + n] = nums[i]`.
6. Return `ans`.

TypeScript solution:

```
class Solution {
  getConcatenation(nums: number[]): number[] {
    const n = nums.length;
    const ans = new Array<number>(2 * n);

    for (let i = 0; i < n; i++) {
      ans[i] = nums[i];
      ans[i + n] = nums[i];
    }

    return ans;
  }
}
```

Complexity:

- Time: $O(n)$
- Space: $O(n)$ (output array of size `2n`)

Stacks

A stack is a data structure that supports a subset of dynamic-array operations.

You can only add and remove elements from one end, called the top. In practice, stacks follow **LIFO (Last In, First Out)** behavior: the last element inserted is the first one removed.

An intuitive analogy is a stack of plates. You can only place a plate on top or take one from top.

Core stack operations are:

- push (insert at top)
- pop (remove from top)
- peek (read top without removing)

Push

Push inserts an element at the top of the stack. With a dynamic array, this is append-at-end and is typically $O(1)$.

```
push(n) {
    this.stack.push(n);
}
```

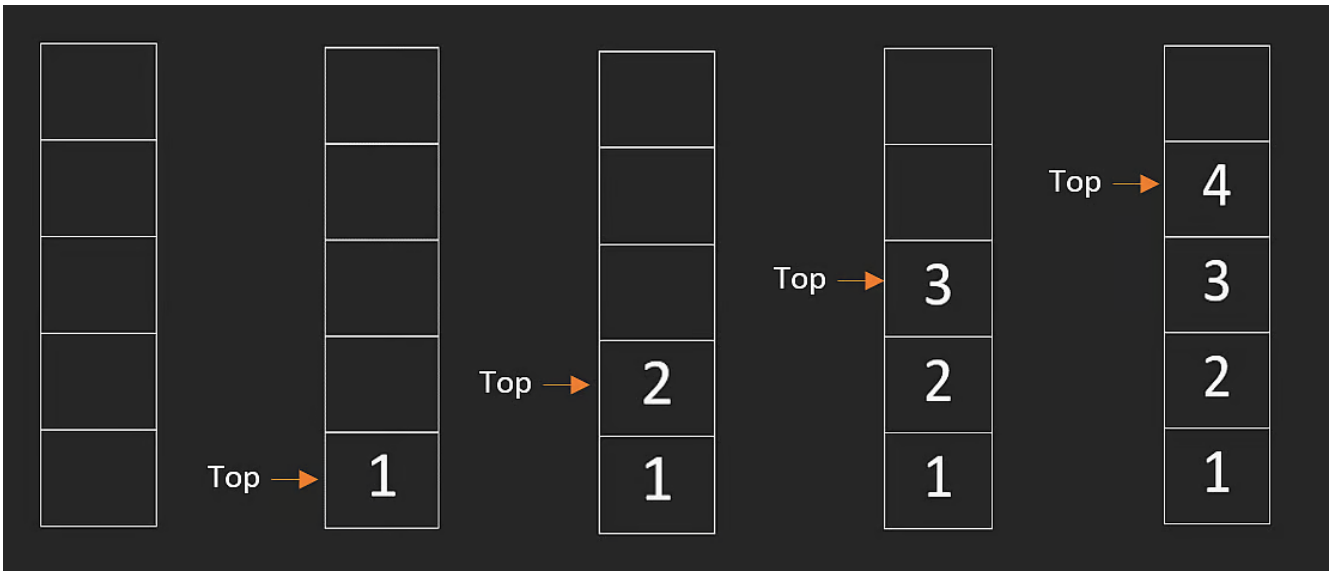


Figure 11: Stack push operation

In many languages, stacks are implemented directly with dynamic arrays.

Pop

Pop removes and returns the top element. With a dynamic array, this is remove-last and is typically $O(1)$.

```
pop() {
    return this.stack.pop();
}
```

Peek

Peek returns the current top element without removing it.

```
peek() {
    return this.stack[this.stack.length - 1];
}
```

This operation is also $O(1)$.

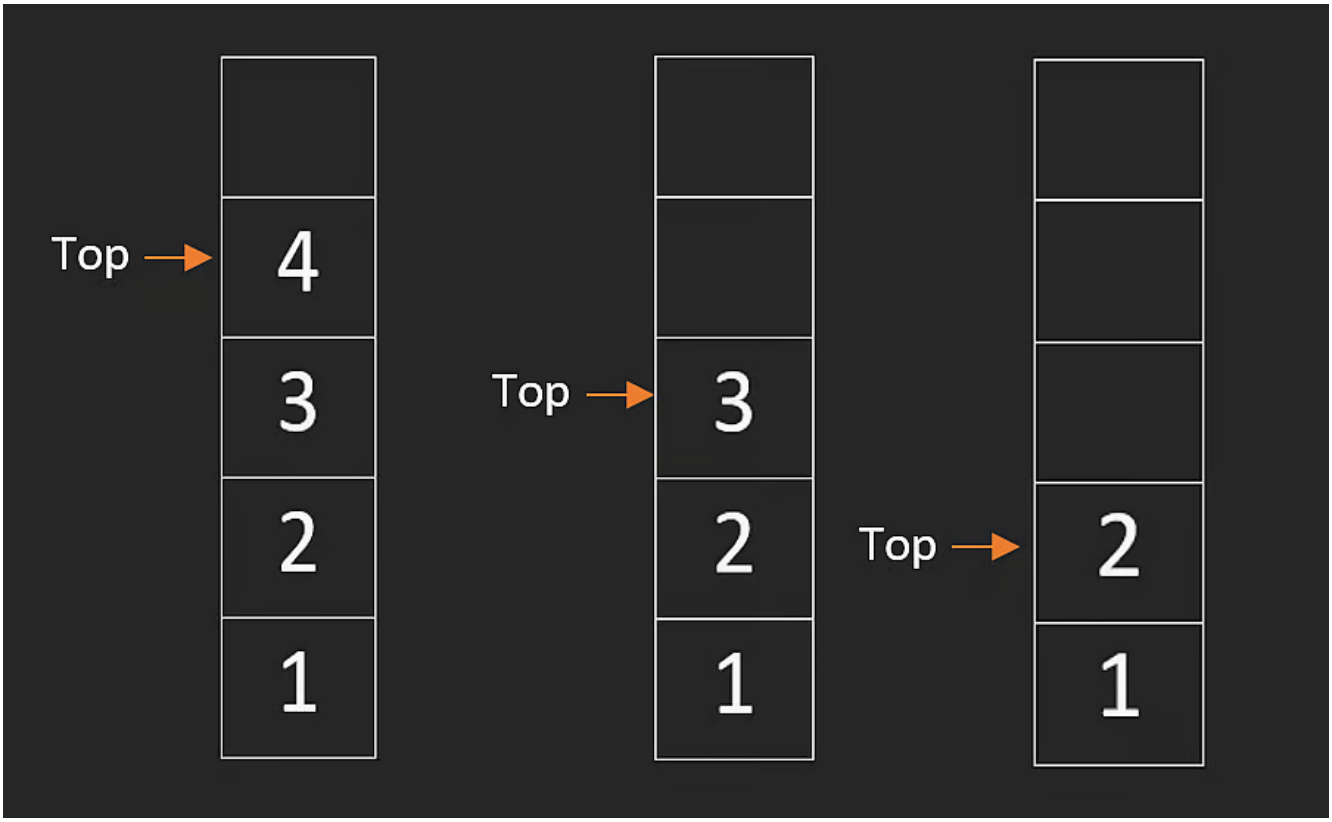


Figure 12: Stack pop operation

Stack Time Complexity Summary

Time Complexity

Operation	Big-O Time Complexity	Notes
Push	$O(1)$	
Pop	$O(1)^*$	Check if the stack is empty first
Peek / Top	$O(1)^*$	Retrieves without removing

Figure 13: Time complexity for stack operations

NeetCode Practice (Stacks)

1) Baseball Game

- Difficulty: **Easy**
- Pattern: **Simulation with stack**
- Core idea: Use a stack to keep valid scores. Process each operation and maintain a running sum.

Problem statement (summary): Given a list of operations, process baseball scoring rules:

- Integer `x`: add score `x`
- `+`: add sum of previous two valid scores
- `D`: add double previous valid score
- `C`: remove previous valid score

Return total score after all operations.

Example:

- Input: `ops = ["1","2","+","C","5","D"]`
- Output: `18`

Intuition: The operations only depend on recent valid scores, so a stack is ideal. Keeping a running total avoids an extra pass at the end.

Algorithm:

1. Initialize empty stack and `res = 0`.
2. For each operation:
3. If `+`, sum top two scores, push result, add to `res`.
4. If `D`, double top score, push result, add to `res`.
5. If `C`, pop top score and subtract from `res`.
6. Otherwise parse number, push it, add to `res`.
7. Return `res`.

TypeScript solution:

```
class Solution {
  calPoints(operations: string[]): number {
    const stack: number[] = [];
    let res = 0;

    for (const op of operations) {
      if (op === "+") {
        const top = stack.pop() as number;
        const newTop = top + stack[stack.length - 1];
        stack.push(top);
        stack.push(newTop);
        res += newTop;
      } else if (op === "D") {
```

```
        const doubled = 2 * stack[stack.length - 1];
        stack.push(doubled);
        res += doubled;
      } else if (op === "C") {
        res -= stack.pop() as number;
      } else {
        const score = Number(op);
        stack.push(score);
        res += score;
      }
    }

    return res;
  }
}
```

Complexity:

- Time: $O(n)$
- Space: $O(n)$

2) Valid Parentheses

- Difficulty: **Easy**
- Pattern: **Matching pairs with stack**
- Core idea: Push opening brackets. On closing bracket, it must match the most recent opening bracket.

Problem statement (summary): Given a string containing only `()[]{}`, return `true` if all brackets are correctly matched and nested.

Examples:

- Input: `s = "[]"` -> Output: `true`
- Input: `s = "([{}])"` -> Output: `true`
- Input: `s = "[(])"` -> Output: `false`

Intuition: Valid nesting requires last-opened to be first-closed, which is exactly stack behavior.

Algorithm:

1. Create stack for opening brackets.
2. Create map from closing to opening bracket.
3. Iterate characters in `s`.
4. If opening bracket, push it.
5. If closing bracket, check top matches mapped opener.
6. If not matched, return `false`; otherwise pop.
7. At end, return whether stack is empty.

TypeScript solution:

```typescript
class Solution {
  isValid(s: string): boolean {
    const stack: string[] = [];
    const closeToOpen: Record<string, string> = {
      ")": "(",
      "]": "[",
      "}": "{",
    };

    for (const c of s) {
      if (c in closeToOpen) {
        if (stack.length > 0 && stack[stack.length - 1] === closeToOpen[c]) {
          stack.pop();
        } else {
          return false;
        }
      } else {
        stack.push(c);
      }
    }

    return stack.length === 0;
  }
}
```

Complexity:

- Time: $O(n)$
- Space: $O(n)$

3) Min Stack

- Difficulty: **Medium**
- Pattern: **Stack with min tracking**
- Core idea: Store encoded differences in one stack and maintain current minimum in a variable.

Problem statement (summary): Design a stack supporting:

- `push(val)`
- `pop()`
- `top()`
- `getMin()`

All operations must be $O(1)$.

Example:

- Input: `["MinStack", "push", 1, "push", 2, "push", 0, "getMin", "pop", "top", "getMin"]`
- Output: `[null, null, null, null, 0, null, 2, 1]`

Constraints:

- $-2^{31} \leq val \leq 2^{31} - 1$
- `pop`, `top`, and `getMin` are called only on non-empty stack

Intuition: Store `val - min` instead of raw values. Negative encoded values signal that a new minimum was introduced. On pop, a negative value lets us restore the previous minimum in constant time.

Algorithm:

1. Track `stack` (encoded values) and `min` (current minimum).
2. On first push, store `0` and set `min = val`.
3. On next pushes, store `val - min`; if `val < min`, update `min`.
4. On pop, if encoded value is negative, restore old minimum.
5. On top, decode using current `min`.
6. On getMin, return current `min`.

TypeScript solution:

```typescript
class MinStack {
  private min: number;
  private stack: number[];

  constructor() {
    this.min = Infinity;
    this.stack = [];
  }

  push(val: number): void {
    if (this.stack.length === 0) {
      this.stack.push(0);
      this.min = val;
    } else {
      this.stack.push(val - this.min);
      if (val < this.min) {
        this.min = val;
      }
    }
  }

  pop(): void {
    if (this.stack.length === 0) {
      return;
```

```
    }

    const encoded = this.stack.pop() as number;
    if (encoded < 0) {
      this.min -= encoded;
    }
  }

  top(): number {
    const encoded = this.stack[this.stack.length - 1];
    return encoded > 0 ? encoded + this.min : this.min;
  }

  getMin(): number {
    return this.min;
  }
}
```

Complexity:

- Time: $O(1)$ for all operations
- Space: $O(n)$

Singly Linked Lists

A linked list stores elements in an ordered sequence, similar to an array. The key difference is that elements are **not** stored contiguously in memory.

Instead, a linked list is made up of objects called **ListNodes**, each with two attributes:

- `value` — the data stored in the node (a number, string, etc.)
- `next` — a reference to the next node in the list

Creating a Linked List

We start by defining the `ListNode` class:

```
class ListNode {
    constructor(value) {
        this.value = value;
        this.next = null;
    }
}
```

By chaining nodes together via `next`, we build a list. Suppose we have three nodes: `ListNode1`, `ListNode2`, `ListNode3`.

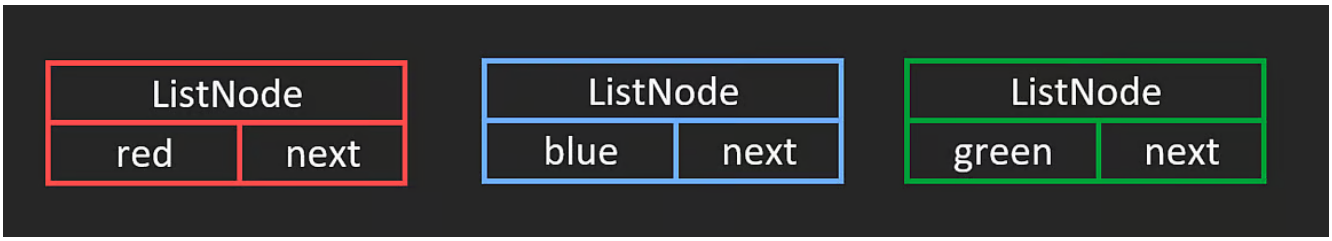


Figure 14: Singly linked list

In this example, values are strings — red, blue, green.

When nodes are instantiated, they are likely **not** stored contiguously in memory (unlike arrays). This is fine — the `next` pointer connects them regardless of location.

To connect the nodes:

```
ListNode1.next = ListNode2;
ListNode2.next = ListNode3;
ListNode3.next = null; // last node always points to null
```

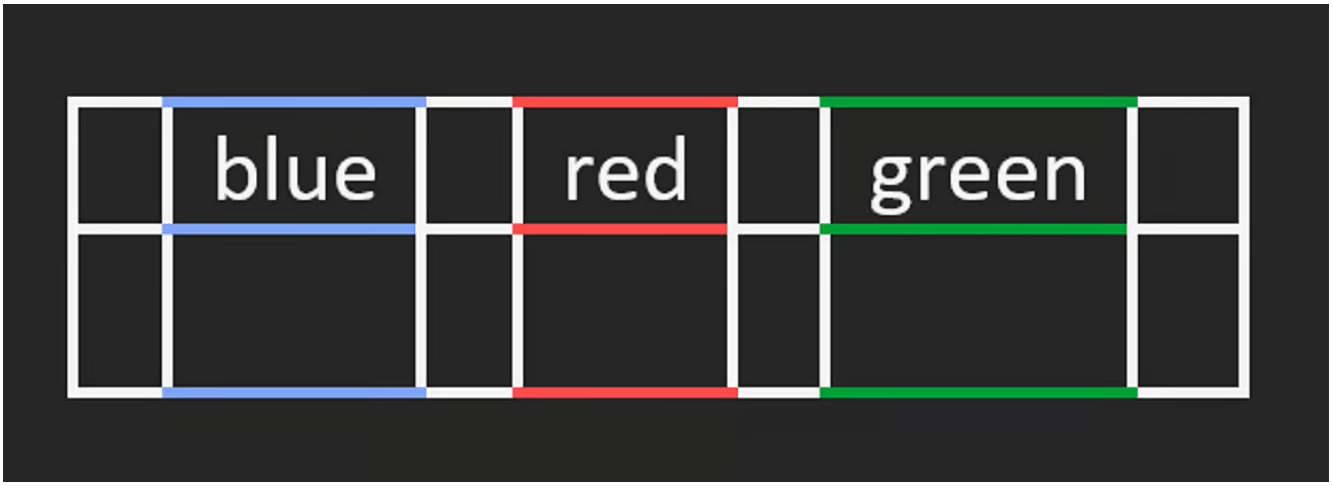


Figure 15: Singly linked list in memory

Traversal

To traverse from start to end, use a `while` loop:

```
let cur = ListNode1;
while (cur) {
    cur = cur.next;
}
```

1. Start at the `head` of the list.
2. Track position in `cur`.
3. Loop until `cur` becomes `null`.
4. Each iteration: advance via `cur = cur.next`.

- Time complexity: $O(n)$

Circular Linked List

If the last node's `next` points back to the first node instead of `null`, the result is a **circular linked list**.

Iterating through one without a cycle-detection mechanism would cause an *infinite loop*.

Operations

A linked list tracks two pointers: **head** (first node) and **tail** (last node). If there is only one node, both point to the same node.

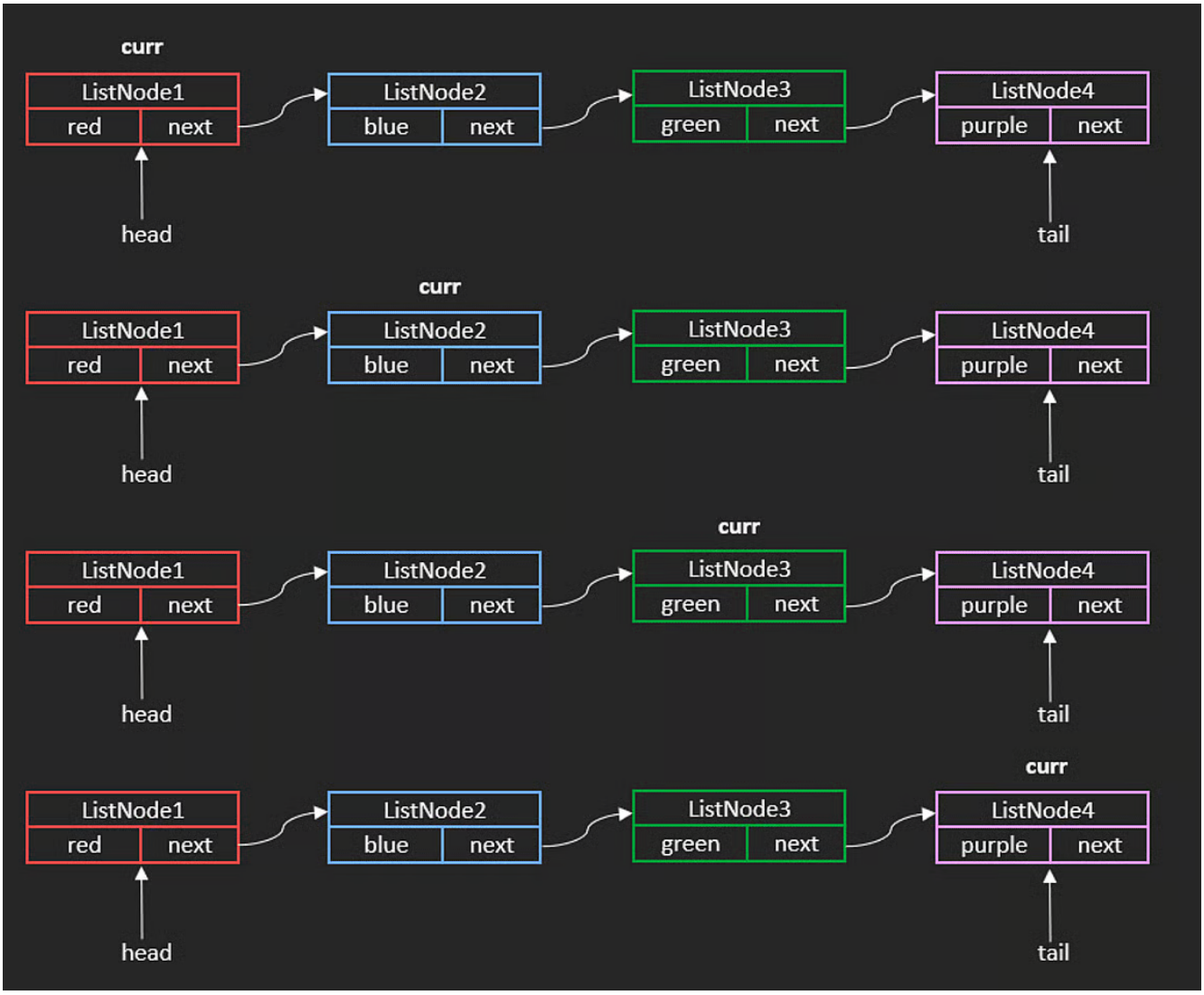


Figure 16: Singly linked list traversal

Appending

Appending to the tail is $O(1)$ — we only update one pointer.

> **Key advantage over arrays:** no shifting of elements is needed, since nodes do not have to be contiguous in memory.
>
> **Note:** This $O(1)$ assumes we already hold a reference to the insertion point. If we have to traverse to find it first, the operation takes $O(n)$.

Steps to append `ListNode4`:

1. Connect the current tail to the new node.

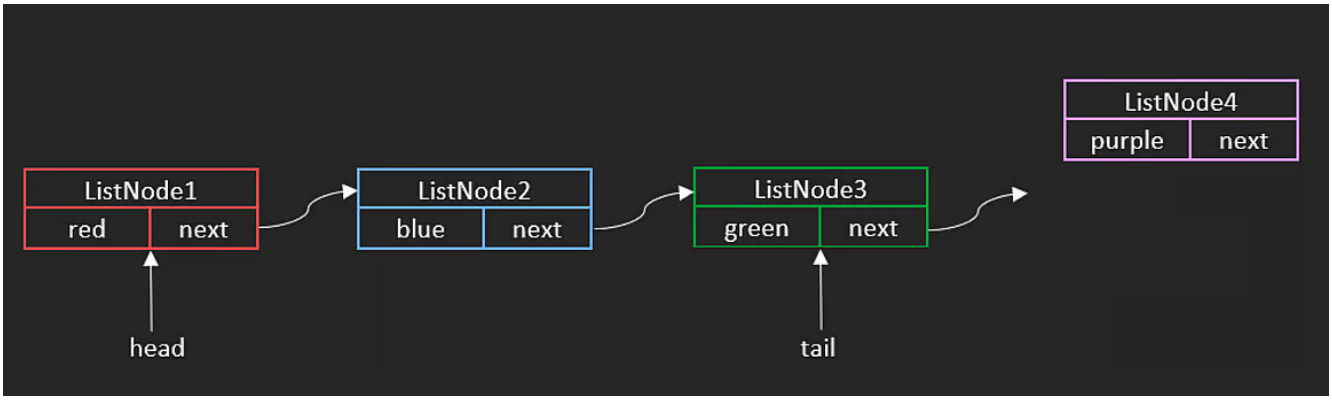


Figure 17: Append step 1

```
tail.next = ListNode4;
```

2. Update the tail pointer.

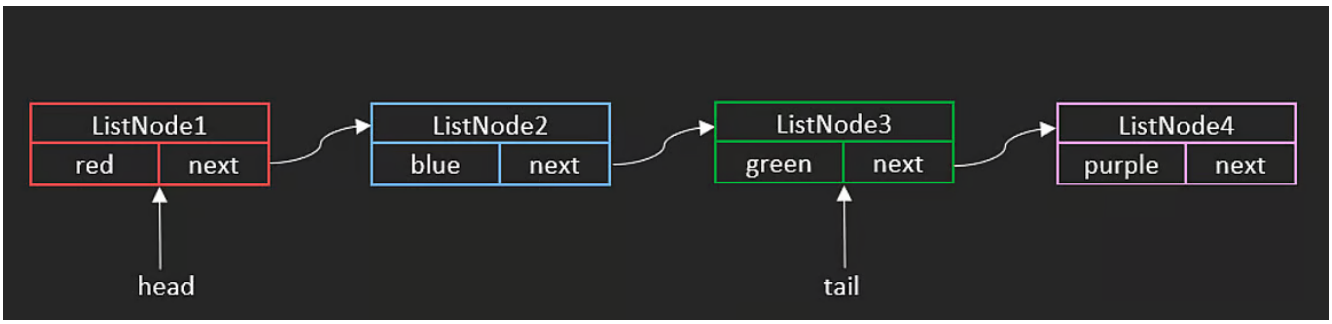


Figure 18: Append step 2

```
tail = ListNode4;
```

Deleting

Deleting a node (given a direct reference) is $O(1)$ — we only update one pointer.

> **Note:** If we have to traverse to find the deletion point first, the operation takes $O(n)$.

To delete `ListNode2`, skip over it by relinking `head.next` to `head.next.next`:

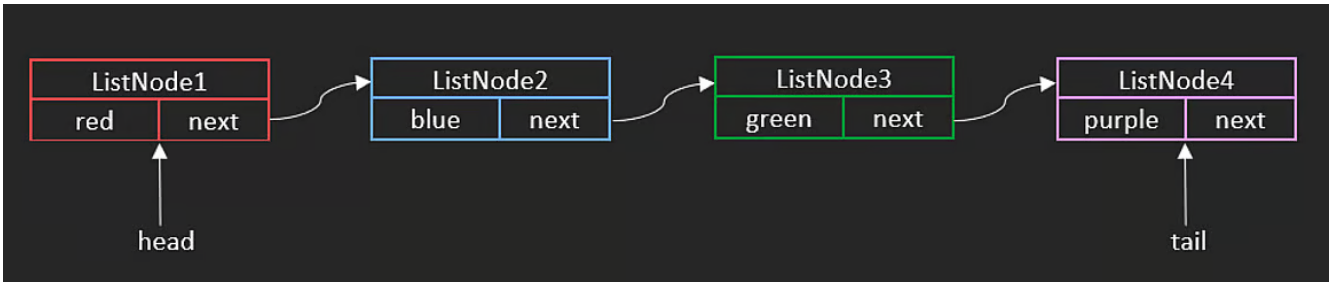


Figure 19: Append step 3 — final state

```
head.next = head.next.next;
```

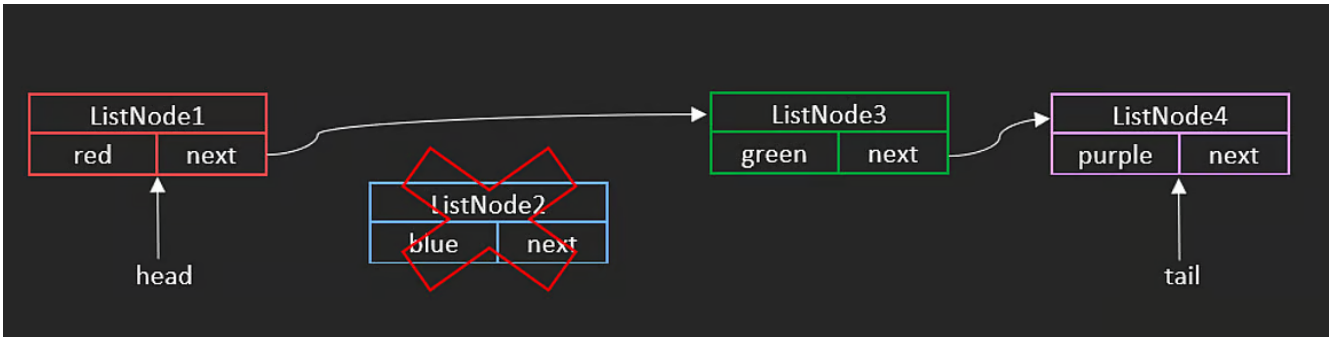


Figure 20: Delete ListNode2

`ListNode1` now points directly to `ListNode3`. In garbage-collected languages, `ListNode2`'s memory is freed automatically. In languages like C, you must free it manually.

Singly Linked List Time Complexity Summary

NeetCode Practice (Singly Linked Lists)

1) Reverse Linked List

- Difficulty: **Easy**
- Pattern: **Pointer reversal**
- Core idea: Flip every `next` pointer to face backward. Two approaches: iterative (preferred for space) or recursive.

Problem statement (summary): Given the head of a singly linked list, reverse the list and return the new head.

Example:

- Input: `head = [0,1,2,3]`
- Output: `[3,2,1,0]`

Time Complexity

Operation	Big-O Time Complexity	Note
Access	$O(n)$	
Search	$O(n)$	
Insertion	$O(1)^*$	Assuming you already have a reference to the node at the desired position
Deletion	$O(1)^*$	Assuming you already have a reference to the node at the desired position

Figure 21: Time complexity for singly linked list

Approach 1: Iterative **Intuition:** Walk left-to-right, flipping each `next` pointer backward at every step. Three pointers track the current node, the previous node, and the saved-next so the chain is not lost.

Algorithm:

1. Set `prev = null`, `curr = head`.
2. While `curr` exists:
3. Save `temp = curr.next`.
4. Flip: `curr.next = prev`.
5. Advance: `prev = curr`, `curr = temp`.
6. Return `prev` (new head).

TypeScript solution:

```typescript
interface ListNode {
    val: number;
    next: ListNode | null;
}

class Solution {
    reverseList(head: ListNode | null): ListNode | null {
        let prev: ListNode | null = null;
        let curr: ListNode | null = head;

        while (curr) {
            const temp = curr.next;
            curr.next = prev;
            prev = curr;
            curr = temp;
        }

        return prev;
    }
}
```

Complexity:

- Time: $O(n)$
- Space: $O(1)$

Approach 2: Recursive **Intuition:** Recurse all the way to the last node — it becomes the new head. On the way back up the call stack, reverse one link per frame, then break the original forward link to avoid cycles.

```
reverseList(0)
  -> reverseList(1)
    -> reverseList(2)
      -> reverseList(3)   // base case: returns node 3 as newHead
    // unwind: 3 -> 2
  // unwind: 2 -> 1
// unwind: 1 -> 0
```

Algorithm:

1. Base case: if `head` is `null`, return `null`.
2. Recurse on `head.next`; save the returned new head.
3. Reverse current link: `head.next.next = head`.
4. Break forward link: `head.next = null`.
5. Return the new head (original last node).

TypeScript solution:

```
class Solution {
    reverseList(head: ListNode | null): ListNode | null {
        if (!head) return null;

        let newHead: ListNode = head;

        if (head.next) {
            // Recurse; newHead ends up as the original last node.
            newHead = this.reverseList(head.next) as ListNode;

            // Reverse the current link (e.g. 2 -> 3 becomes 3 -> 2).
            head.next.next = head;
        }

        // Break the original forward link to avoid a cycle.
        head.next = null;

        return newHead;
    }
}
```

Complexity:

- Time: $O(n)$
- Space: $O(n)$ — recursive call stack

Merge Two Sorted Linked Lists Easy Topics Company Tags Hints You are given the heads of two sorted linked lists list1 and list2.

Merge the two lists into one sorted linked list and return the head of the new sorted linked list.

The new list should be made up of nodes from list1 and list2.

Input: list1 = [1,2,4], list2 = [1,3,5]

Output: [1,1,2,3,4,5]

1. Recursion Intuition Merging two sorted linked lists recursively works by always choosing the smaller head node of the two lists. Whichever list has the smaller value should appear first in the merged list. So we:

Pick the smaller node. Recursively merge the rest of the lists. Attach the result to the chosen node. Algorithm If one list is empty, return the other list — nothing left to merge. Compare the head values of list1 and list2: If list1.val <= list2.val: Set list1.next to the merged result of the remaining nodes. Return list1 as the current head. Otherwise: Set list2.next to the merged result of the remaining nodes. Return list2 as the current head. The recursion continues until both lists are fully merged.

mergeTwoLists(list1: ListNode | null, list2: ListNode | null): ListNode { /* Input: list1 = [1,2,4], list2 = [1,3,5]

```
    RECURSION
    */

    if (!list1) {
        return list2
    }

    if (!list2) {
        return list1
    }

    if (list1.val <= list2.val) {
        list1.next = this.mergeTwoLists(list1.next, list2)
        return list1;
    } else {
        list2.next = this.mergeTwoLists(list1, list2.next)
        return list2;
    }
```

}

Time complexity: O(n+m) Space complexity: O(n+m)

2. Iteration Intuition To merge two sorted linked lists iteratively, we build the result step-by-step. We keep a pointer (node) to the current end of the merged list, and at each step we choose the smaller head node from list1 or list2.

Because the lists are already sorted, whichever head is smaller must come next in the merged list. We attach that node, move the pointer forward, and continue until one list is empty. Finally, we attach the remaining nodes from the non-empty list.

Using a dummy node makes handling the head of the merged list simple and clean.

Algorithm Create a dummy node and a node pointer pointing to it. While both lists have nodes: Compare list1.val and list2.val. Attach the smaller node to node.next. Move forward in the chosen list. Move node to node.next. When one list becomes empty: Attach the remaining nodes of the other list to node.next. Return dummy.next, which is the head of the merged list.

mergeTwoLists(list1: ListNode | null, list2: ListNode | null): ListNode {

```
    const dummy = { val: 0, next: null };
    let node = dummy;

    while (list1 && list2) {
        if (list1.val < list2.val) {
            node.next = list1;
            list1 = list1.next;
        } else {
            node.next = list2;
            list2 = list2.next;
        }
        node = node.next;
    }

    if (list1) {
        node.next = list1;
    } else {
        node.next = list2;
    }

    return dummy.next;
}
```

Time complexity: O(n+m) Space complexity: O(1) because we are not using any additional data structures that scale with input size.

Doubly Linked Lists

After learning singly linked lists, the next variation is the **doubly linked list**.

Each node has two pointers:

- `next`: points to the next node
- `prev`: points to the previous node

If `prev` is `null`, that indicates we are at the head of the list.

Insertion at the End

Like singly linked lists, inserting at the tail is $O(1)$ when we already track `tail`.

The difference is that we also update the `prev` pointer of the new node.

Example with `ListNode1 -> ListNode2 -> ListNode3`, inserting `ListNode4` at the end:

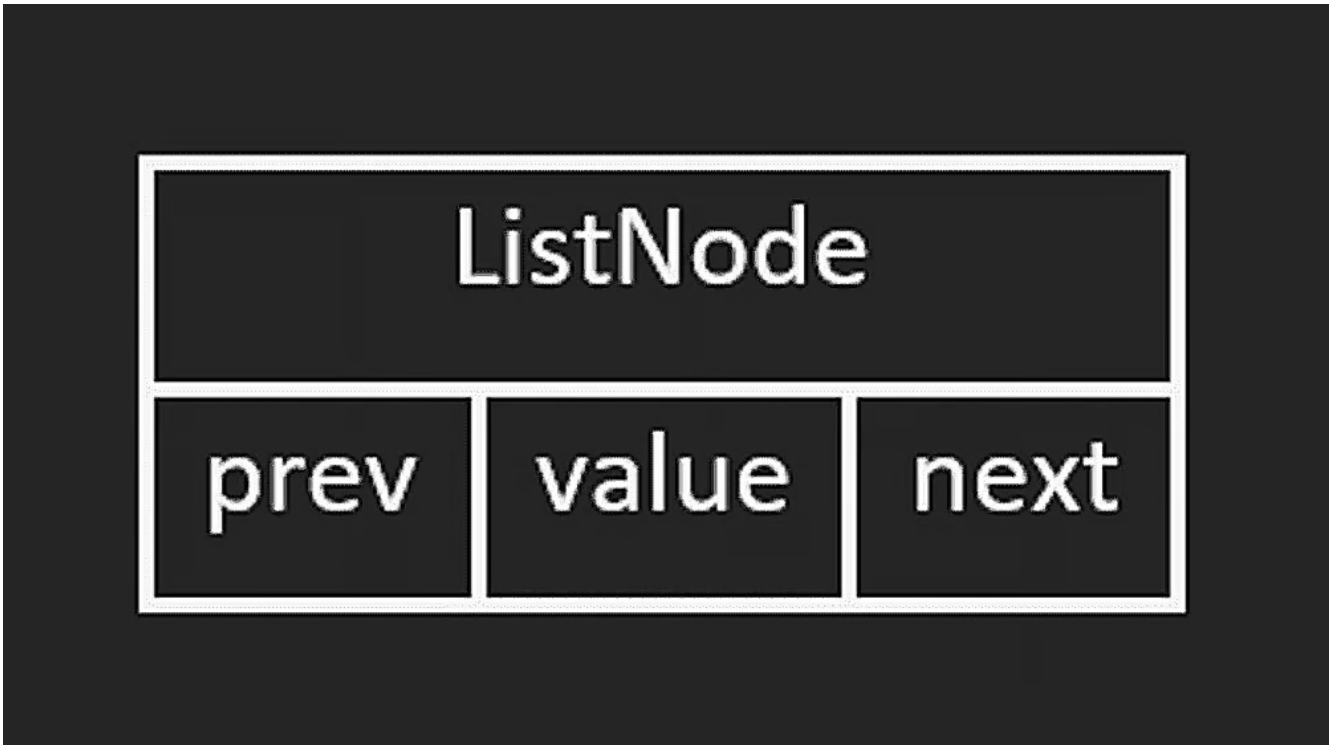


Figure 22: Doubly linked list structure

```
tail.next = ListNode4;
ListNode4.prev = tail;
tail = tail.next;
```

Deletion at the End

Deleting at the tail is also $O(1)$.

Steps:

1. Get reference to the node before tail (`tail.prev`).
2. Set its `next` pointer to `null`.
3. Move `tail` to that previous node.
4. Optionally clear the old tail's `prev`.

```
ListNode2 = tail.prev;
ListNode2.next = null;
tail = ListNode2;
```

Because both insertion and deletion at one end are $O(1)$, a linked list can theoretically implement stack behavior as well (though arrays are often more common in practice).

Access Pattern

As with singly linked lists, random access is not possible.

In the worst case, reaching a specific node requires traversing up to `n` nodes:

- Access by index/value search: $O(n)$

The advantage over singly linked lists is that traversal can happen in both directions.

Time Complexity Summary

Operation	Big-O Time Complexity	Notes
Access	$O(n)$	
Search	$O(n)$	
Insertion	$O(1)^*$	Assuming you have the reference to the node at the desired position
Deletion	$O(1)^*$	Assuming you have the reference to the node at the desired position

Figure 23: Doubly linked list time complexity

Queues

A **queue** is a data structure with **FIFO (First In, First Out)** behavior.

It is similar to stacks in being linear, but different in removal order:

- Stack: Last In, First Out
- Queue: First In, First Out

A real-world example is a line at a bank: the first person in line is the first person served.

Implementation and Core Operations

The easiest queue implementation is with a linked list.

You can also implement it with a dynamic array, but to match linked-list efficiency you generally need a **circular array** and extra index management.

Core queue operations:

- `enqueue`: add at the end (tail)
- `dequeue`: remove from the front (head)

Enqueue

`enqueue` inserts a value at the end of the queue. With a singly linked list and maintained head/tail references, this is $O(1)$.

```
enqueue(val) {
    const newNode = new ListNode(val);

    if (this.right != null) {
        // Queue is not empty
        this.right.next = newNode;
        this.right = this.right.next;
    } else {
        // Queue is empty
        this.left = newNode;
        this.right = newNode;
    }
}
```

Dequeue

`dequeue` removes and returns the value from the front of the queue.

```
dequeue() {
    if (this.left == null) {
        // Queue is empty
        return;
    }

    // Remove left node and return value
    const val = this.left.val;
    this.left = this.left.next;

    if (!this.left) {
        this.right = null;
    }

    return val;
}
```

As with stacks, it is good practice to handle empty-structure checks before removals.

Deque Variation

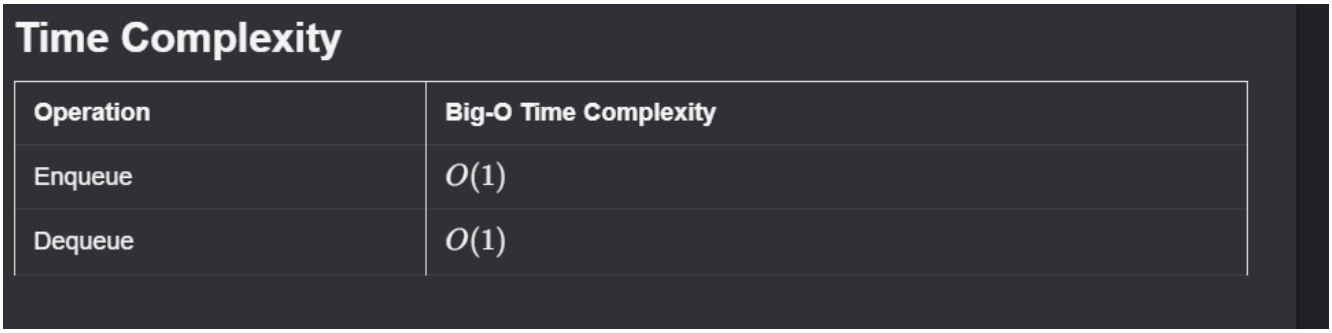
A **deque** (pronounced "deck") is a double-ended queue.

It allows insertion/removal from both head and tail in $O(1)$.

Common Use Case

Queues are fundamental for **breadth-first search (BFS)** in trees and graphs.

Time Complexity Summary



Time Complexity

Operation	Big-O Time Complexity
Enqueue	$O(1)$
Dequeue	$O(1)$

Figure 24: Queue time complexity

Recursion (One Branch)

Recursion can feel unintuitive at first, so if it does not click immediately, that is normal.

At a high level, recursion means a function calls itself, usually with a smaller or simpler input.

A recursive function has two required parts:

- **Base case**: the stopping condition
- **Recursive case**: the part where the function calls itself

You can often convert recursive solutions to iterative ones, and vice versa.

Intuition

Recursion breaks one problem into smaller sub-problems, then combines those answers while unwinding back up.

A real-world analogy is a box inside another box, inside another box:

- smallest box -> base case
- outer boxes -> recursive cases

There are two common recursion patterns:

- one-branch recursion
- multi-branch recursion

This lesson focuses on one-branch recursion.

Factorial Example

To compute $n!$:

$$n! = n \cdot (n-1) \cdot (n-2) \cdots 1$$

Example:

$$5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 120$$

Key recursive relationship:

- $5! = 5 \cdot 4!$
- $4! = 4 \cdot 3!$
- $\ldots$

Base case:

- $0! = 1$
- $1! = 1$

```
// Recursive implementation of n! (n-factorial)
function factorial(n) {
  // Base case
  if (n <= 1) {
    return 1;
  }

  // Recursive case: n! = n * (n - 1)!
  return n * factorial(n - 1);
}
```

Walkthrough: `factorial(5)`

Calls go down first:

- `factorial(5)` -> `5 * factorial(4)`
- `factorial(4)` -> `4 * factorial(3)`
- `factorial(3)` -> `3 * factorial(2)`
- `factorial(2)` -> `2 * factorial(1)`
- `factorial(1)` -> `1` (base case)

Then results come back up:

- `factorial(2) = 2 * 1 = 2`
- `factorial(3) = 3 * 2 = 6`
- `factorial(4) = 4 * 6 = 24`
- `factorial(5) = 5 * 24 = 120`

This upward phase is where sub-problem answers are combined.

If a recursion has no base case, it keeps calling forever and eventually causes stack overflow.

Time and Space Complexity

- Time: $O(n)$
- Space: $O(n)$

Reasoning:

- There are `n` calls.
- Each call does $O(1)$ extra work.
- Recursion uses the implicit function call stack, which stores up to `n` frames.

Iteration vs Recursion

Most recursive solutions can also be written iteratively.

```
let n = 5;
let res = 1;

while (n > 1) {
  res = res * n;
  n--;
}
```

Here we update `res` while decrementing `n` until `n` becomes `1`.

For factorial specifically, the iterative version is simpler and more space efficient because it does not use the call stack.

Recursion becomes especially useful for tree-style problems, where the recursive structure closely matches the data structure.

NeetCode Practice (Recursion)

1) Reverse Linked List

- Difficulty: **Easy**
- Pattern: **Recursive pointer reversal**
- Core idea: Reverse the rest of the list first, then point the next node back to current.

Problem statement (summary): Given the head of a singly linked list, reverse the list and return the new head.

Example:

- Input: `head = [0,1,2,3]`
- Output: `[3,2,1,0]`

Intuition: Go all the way to the end recursively. The last node becomes the new head. While recursion unwinds, flip each `next` pointer backward, and finally set old head's `next` to `null`.

Algorithm:

1. If `head` is null, return `null`.
2. Recursively reverse `head.next`.
3. When returning, set `head.next.next = head`.
4. Set `head.next = null` to avoid cycle.
5. Return the new head from the deepest recursive call.

TypeScript solution:

```
/**
 * Definition for singly-linked list.
 * class ListNode {
 *     constructor(val = 0, next = null) {
 *         this.val = val;
 *         this.next = next;
 *     }
 * }
 */

class Solution {
  reverseList(head: ListNode | null): ListNode | null {
    if (!head) {
      return null;
    }

    let newHead = head;
    if (head.next) {
      newHead = this.reverseList(head.next);
      head.next.next = head;
    }

    head.next = null;
    return newHead;
  }
}
```

Complexity:

- Time: $O(n)$
- Space: $O(n)$ (recursion stack)

Recursion (Two Branch)

A common form of recursion is **multi-branch recursion**.

In this lesson, we use the Fibonacci sequence as a classic two-branch example.

Fibonacci Sequence

The Fibonacci sequence starts as:

`0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...`

Each value is the sum of the previous two values.

Formally:

- $F(0) = 0$
- $F(1) = 1$
- $F(n) = F(n-1) + F(n-2)$

Base cases are $F(0)$ and $F(1)$. Recursive case is $F(n) = F(n-1) + F(n-2)$.

Recurrence relation:

$$fib(n) = fib(n-1) + fib(n-2)$$

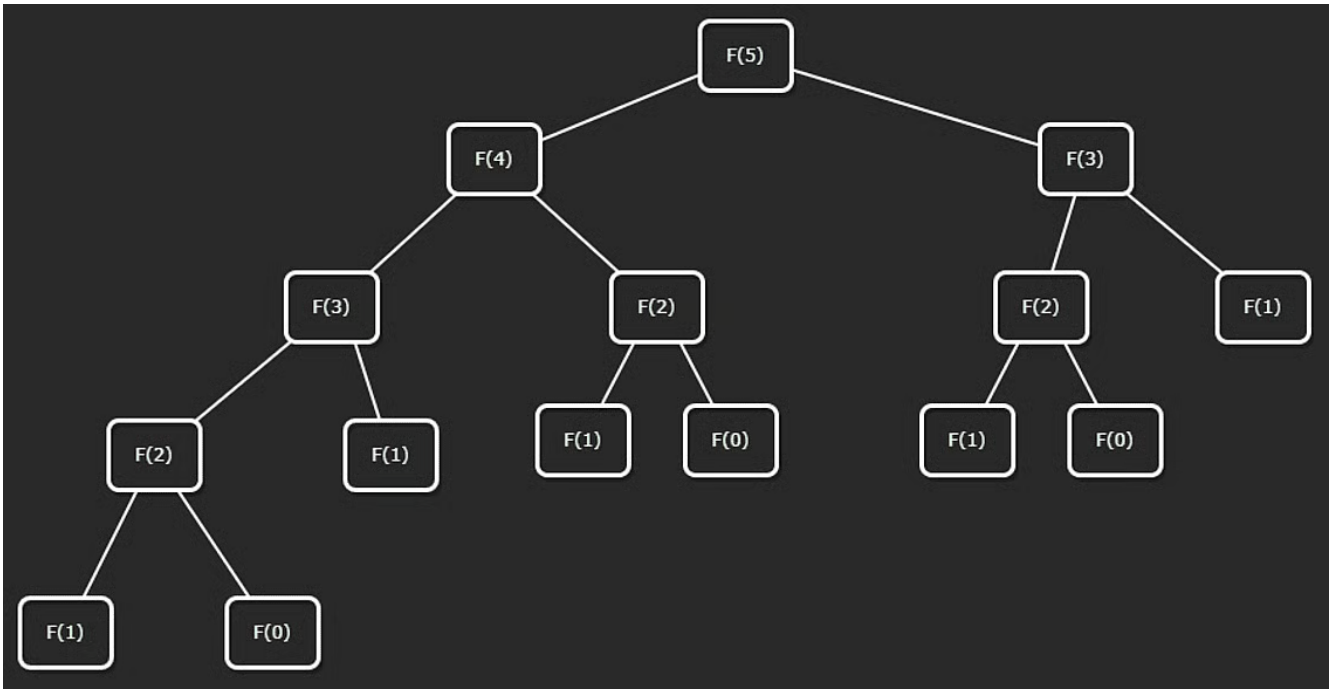


Figure 25: Fibonacci recursion tree

```
// Recursive implementation to calculate the n-th Fibonacci number
function fibonacci(n) {
  // Base case
  if (n <= 1) {
    return n;
  }

  // Recursive case
  return fibonacci(n - 1) + fibonacci(n - 2);
}
```

Walkthrough: `fibonacci(5)`

Start with:

- `fibonacci(5) = fibonacci(4) + fibonacci(3)`

Each branch recursively expands until base cases (`0` or `1`) are reached.

Then values are combined while returning back up:

- `fibonacci(4) = 3`
- `fibonacci(3) = 2`
- `fibonacci(5) = 3 + 2 = 5`

Complexity of Naive Recursion

At each level, calls can branch by about 2x.

An upper-bound node count follows a geometric sum:

$$1 + 2 + 4 + 8 + \cdots + 2^n$$

So naive recursive Fibonacci has exponential time:

- Time: $O(2^n)$
- Space: $O(n)$ recursion depth

NeetCode Practice (Recursion to DP)

1) Climbing Stairs

- Difficulty: **Easy**
- Pattern: **Recursion -> Dynamic Programming**
- Core idea: From step `i`, you can move to `i + 1` or `i + 2`, which creates a Fibonacci-like recurrence.

Problem statement (summary): Given `n` steps, and moves of 1 or 2 steps, return the number of distinct ways to reach the top.

Example:

- Input: `n = 2`
- Output: `2`
- Explanation: `1 + 1`, `2`

Approach 1: Top-Down DP (Memoization)

Intuition: Pure recursion repeats the same subproblems many times. Memoization stores each result once and reuses it.

Algorithm:

1. Create `cache` initialized to `-1`.
2. Define `dfs(i)`:
3. If `i == n`, return `1`.
4. If `i > n`, return `0`.
5. If cached, return cache value.
6. Compute `dfs(i + 1) + dfs(i + 2)`, store, return.
7. Return `dfs(0)`.

```
class Solution {
  climbStairs(n: number): number {
    const cache = new Int32Array(n).fill(-1);

    const dfs = (i: number): number => {
      if (i >= n) {
        return i === n ? 1 : 0;
      }

      if (cache[i] !== -1) {
        return cache[i];
      }

      cache[i] = dfs(i + 1) + dfs(i + 2);
      return cache[i];
    };

    return dfs(0);
  }
}
```

Complexity:

- Time: $O(n)$
- Space: $O(n)$

Approach 2: Bottom-Up DP

Intuition: Ways to reach step `i` come from `i - 1` and `i - 2`, so:

$$dp[i] = dp[i-1] + dp[i-2]$$

Algorithm:

1. If `n <= 2`, return `n`.
2. Build `dp` array of size `n + 1`.
3. Set `dp[1] = 1`, `dp[2] = 2`.
4. Fill from `3` to `n`.
5. Return `dp[n]`.

```
class Solution {
  climbStairs(n: number): number {
    if (n <= 2) {
      return n;
    }

    const dp = new Array<number>(n + 1).fill(0);
    dp[1] = 1;
    dp[2] = 2;

    for (let i = 3; i <= n; i++) {
      dp[i] = dp[i - 1] + dp[i - 2];
    }

    return dp[n];
  }
}
```

Complexity:

- Time: $O(n)$
- Space: $O(n)$

Approach 3: DP (Space Optimized)

Intuition: Only the previous two states are needed, so we can replace the full DP array with two variables.

Algorithm:

1. Initialize `one = 1`, `two = 1`.
2. Repeat `n - 1` times:
3. `newWays = one + two`
4. Shift: `two = one`, `one = newWays`

5. Return one.

```
class Solution {
  climbStairs(n: number): number {
    let one = 1;
    let two = 1;

    for (let i = 0; i < n - 1; i++) {
      const temp = one;
      one = one + two;
      two = temp;
    }

    return one;
  }
}
```

Complexity:

- Time: $O(n)$
- Space: $O(1)$

Insertion Sort

Insertion sort is a classic sorting algorithm used across many data structures.

It is simple and easy to understand, but not the most efficient choice for large datasets.

Concept

Given `[2,3,4,1,6]`, the goal is to produce `[1,2,3,4,6]`.

Insertion sort builds a sorted prefix one element at a time.

Key idea:

- A subarray of size `1` is already sorted.
- Treat the first element as the initial sorted portion.
- Insert each next element into its correct position inside that sorted portion.

Two-pointer view:

- `i`: points to the element being inserted.
- `j`: starts at `i - 1` and moves left.

We keep swapping left while the current element is smaller than the previous element.

```
function insertionSort(arr) {
  for (let i = 1; i < arr.length; i++) {
    let j = i - 1;

    while (j >= 0 && arr[j + 1] < arr[j]) {
      const tmp = arr[j + 1];
      arr[j + 1] = arr[j];
      arr[j] = tmp;
      j--;
    }
  }

  return arr;
}
```

Insertion sort works on any comparable data type, not just integers.

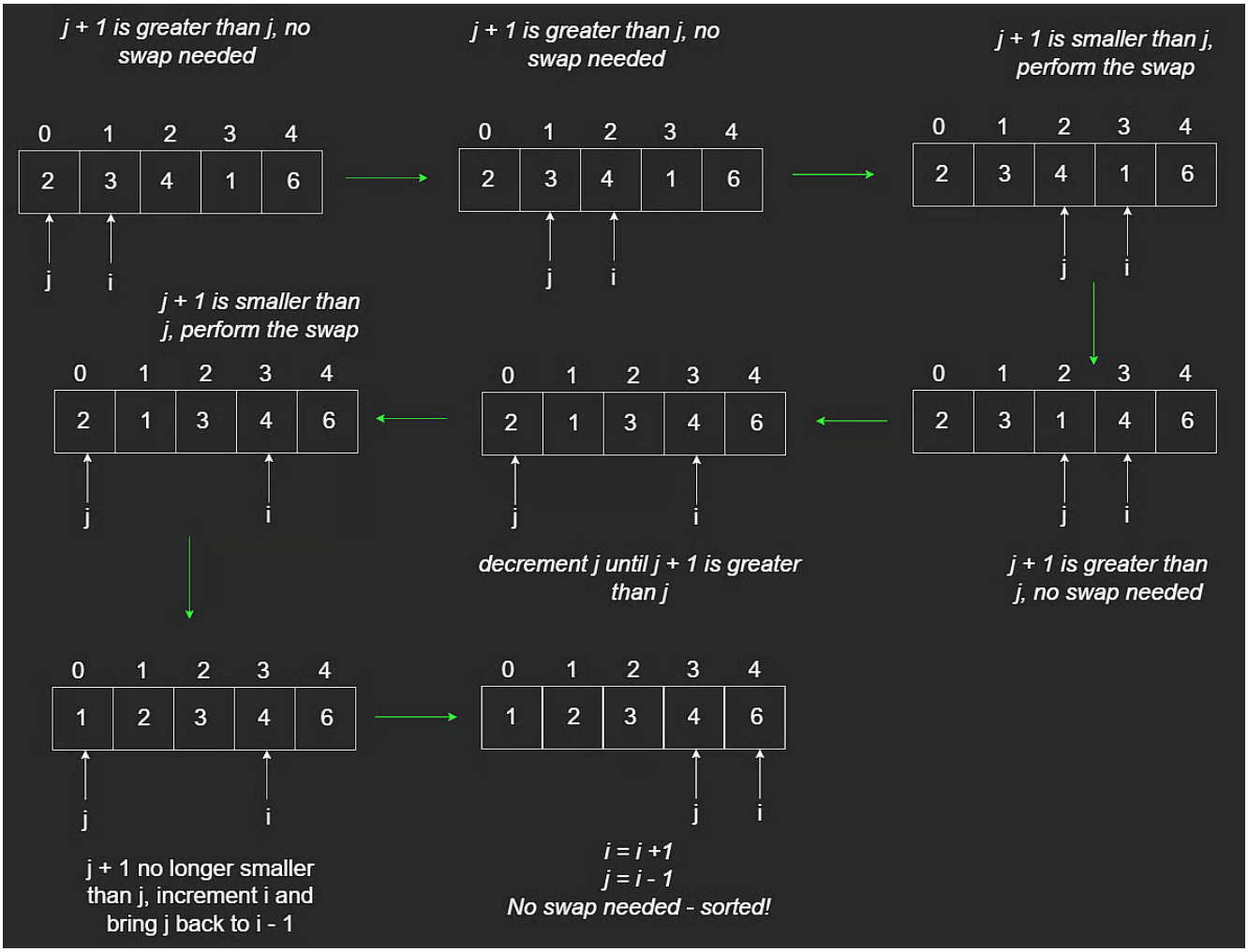


Figure 26: Insertion sort process

Stability

A sorting algorithm is stable if equal values preserve relative order.

Example: `[7,3,7]`.

After sorting, the `7` that originally appeared first still appears before the second `7`.

Insertion sort is stable.

Time and Space Complexity

Time

- Best case (already sorted): $O(n)$
- Average case: $O(n^2)$
- Worst case (reverse sorted, like `[5,4,3,2,1]`): $O(n^2)$

Space

- $O(1)$ (in-place)

NeetCode Practice (Sorting)

1) Insertion Sort (Intermediate States)

- Difficulty: **Easy**
- Pattern: **Stable in-place insertion + snapshots**
- Core idea: Perform insertion sort, and after each outer-loop insertion, save a copy of the current array state.

Problem statement (summary): Given a list of key-value pairs, sort by key using insertion sort and return all intermediate array states.

If two pairs have equal keys, maintain their relative order.

Input constraints:

- `0 <= pairs.length <= 100`
- each pair has integer key and string value

Example:

- Input: `pairs = [(5, "apple"), (2, "banana"), (9, "cherry")]`
- Output:

```
[
  [(5, "apple"), (2, "banana"), (9, "cherry")],
  [(2, "banana"), (5, "apple"), (9, "cherry")],
  [(2, "banana"), (5, "apple"), (9, "cherry")]
]
```

There should be exactly `pairs.length` states.

Algorithm:

1. Initialize result list `res`.
2. For each index `i`, move `pairs[i]` left while previous key is larger.
3. After finishing insertion for this `i`, push a shallow copy of `pairs` into `res`.
4. Return `res`.

TypeScript solution:

```typescript
// class Pair {
//     constructor(public key: number, public value: string) {}
// }

class Solution {
  insertionSort(pairs: Pair[]): Pair[][] {
    const n = pairs.length;
    const res: Pair[][] = [];

    for (let i = 0; i < n; i++) {
      let j = i - 1;

      while (j >= 0 && pairs[j].key > pairs[j + 1].key) {
        [pairs[j], pairs[j + 1]] = [pairs[j + 1], pairs[j]];
        j -= 1;
      }

      res.push([...pairs]);
    }

    return res;
  }
}
```

Complexity:

- Time: $O(n^2)$ in the worst case
- Space: $O(n^2)$ for storing all snapshot states in `res`

Merge Sort

Merge sort is a very common sorting algorithm and is used in many standard libraries.

It follows a **divide-and-conquer** strategy:

1. Split the array into halves until subarrays have size `1`.
2. Recursively sort each half.
3. Merge sorted halves back together.

This is a two-branch recursive pattern, similar to Fibonacci recursion.

Core Idea

Example input: `[3,2,4,1,6]`

Goal: sort in increasing (non-decreasing) order.

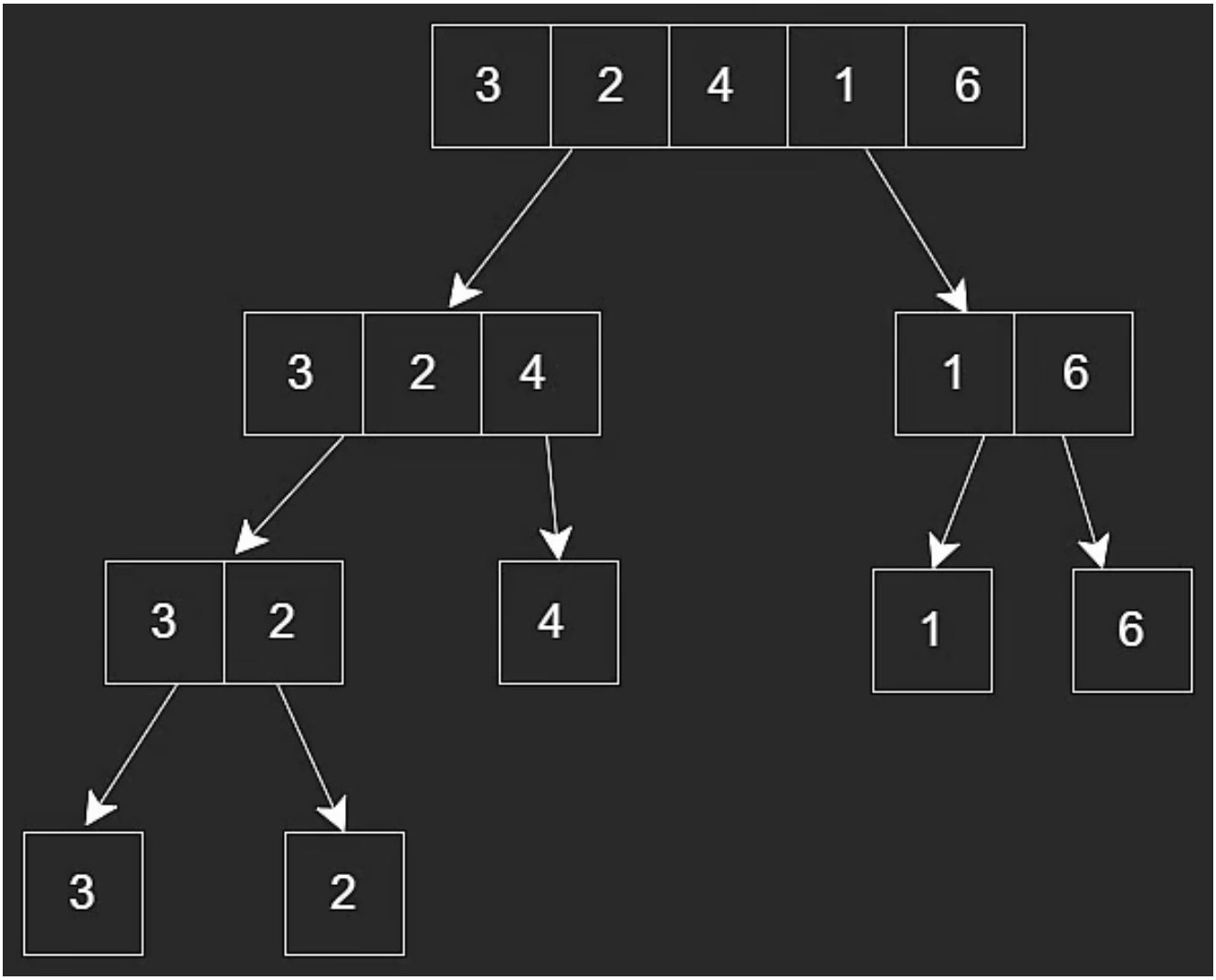


Figure 27: Merge sort split/merge overview

```
function mergeSort(arr, l, r) {
  if (l < r) {
    // Find middle
    const m = Math.floor((l + r) / 2);

    mergeSort(arr, l, m); // sort left half
    mergeSort(arr, m + 1, r); // sort right half
    merge(arr, l, m, r); // merge sorted halves
  }

  return arr;
}
```

Base case: subarray length is `1` (already sorted).

Recursive case:

- sort left half
- sort right half
- merge results

Important execution detail:

- `merge()` runs only after both recursive `mergeSort()` calls return for the current subarray.

Merge Function (Three Pointers)

During merge, we use three pointers:

- `i`: current index in left temporary array
- `j`: current index in right temporary array
- `k`: write index in original array

At each step:

- Compare left and right values
- Write smaller value to `arr[k]`
- Increment `k` and whichever side was chosen (`i` or `j`)

```
// Merges two subarrays of arr[]
// First subarray is arr[l..m]
// Second subarray is arr[m+1..r]
function merge(arr, l, m, r) {
  const length1 = m - l + 1;
  const length2 = r - m;

  const L = new Array(length1);
  const R = new Array(length2);
```

```
  for (let i = 0; i < length1; i++) {
    L[i] = arr[l + i];
  }

  for (let j = 0; j < length2; j++) {
    R[j] = arr[m + 1 + j];
  }

  let i = 0;
  let j = 0;
  let k = l;

  while (i < length1 && j < length2) {
    if (L[i] <= R[j]) {
      arr[k] = L[i];
      i++;
    } else {
      arr[k] = R[j];
      j++;
    }
    k++;
  }

  while (i < length1) {
    arr[k] = L[i];
    i++;
    k++;
  }

  while (j < length2) {
    arr[k] = R[j];
    j++;
    k++;
  }
}
```

This merge step is a classic two-pointer technique and combines arrays in $O(n + m)$ for left length `n` and right length `m`.

Time and Space Complexity

Time

Merge sort runs in $O(n \log n)$.

Reasoning:

- Recursion tree height is about `log n`.
- Each level does `O(n)` work total during merges.
- Multiply: `O(n) * O(log n) = O(n log n)`.

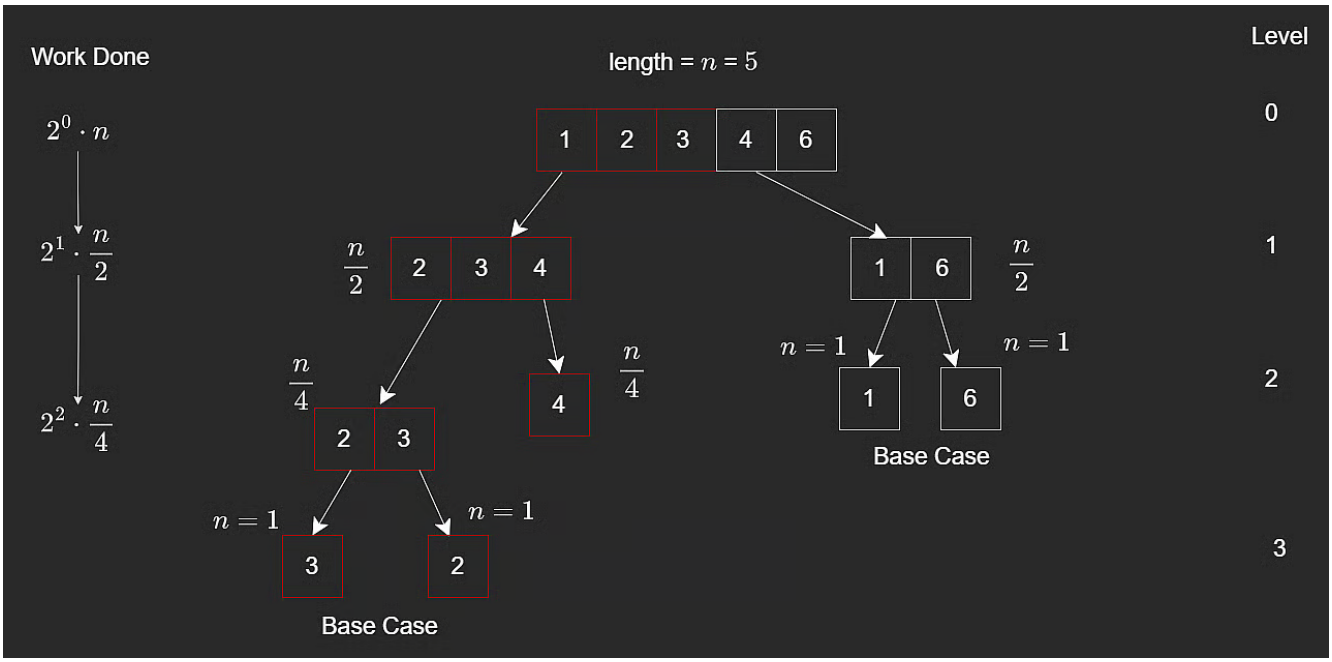


Figure 28: Merge sort recursion levels

Space

- Temporary arrays in merge steps: `O(n)`
- Recursion call stack: `O(log n)`

Total: $O(n + \log n) = O(n)$.

Stability

Merge sort is **stable**.

If duplicates exist in both halves, choosing left first on equality preserves original relative order:

```
if (leftSubarray[i] <= rightSubarray[j]) {
  arr[k] = leftSubarray[i];
  i += 1;
}
```

Merge Sort vs Insertion Sort

- Insertion sort: worst-case $O(n^2)$
- Merge sort: best/average/worst-case $O(n \log n)$

Merge sort is generally superior for larger arrays.

Insertion sort can still be preferable for small or nearly sorted arrays.

NeetCode Practice (Sorting)

1) Merge Sorted Array

- Difficulty: **Easy**
- Pattern: **Two pointers / three pointers**
- Core idea: Merge two sorted arrays into `nums1` in-place.

Problem statement (summary): You are given sorted arrays `nums1` and `nums2`, with counts `m` and `n`.

- First `m` elements of `nums1` are valid.
- Last `n` elements of `nums1` are placeholders (`0`).

Modify `nums1` in-place so it becomes the sorted merge of both arrays.

Example:

- Input: `nums1 = [10,20,20,40,0,0], m = 4, nums2 = [1,2], n = 2`
- Output: `[1,2,10,20,20,40]`

Approach 1: Three Pointers With Extra Space

Intuition: Copy valid portion of `nums1` first so front writes do not overwrite needed values.

Algorithm:

1. Copy first `m` elements of `nums1`.
2. Use `i` for copy, `j` for `nums2`, `idx` for write position in `nums1`.
3. Write smaller value each step.
4. Continue until `idx == m + n`.

```
class Solution {
  merge(nums1: number[], m: number, nums2: number[], n: number): void {
    const nums1Copy = nums1.slice(0, m);
    let idx = 0;
    let i = 0;
    let j = 0;

    while (idx < m + n) {
      if (j >= n || (i < m && nums1Copy[i] <= nums2[j])) {
        nums1[idx++] = nums1Copy[i++];
      } else {
        nums1[idx++] = nums2[j++];
      }
    }
  }
}
```

Complexity:

- Time: $O(m+n)$
- Space: $O(m)$

Approach 2: Three Pointers Without Extra Space (Version I)

Intuition: Write from the back of `nums1` (where free space exists), so no needed value is overwritten.

Algorithm:

1. Set `last = m + n - 1`.
2. While `m > 0` and `n > 0`, place larger of `nums1[m - 1]` and `nums2[n - 1]` at `nums1[last]`.
3. Decrement corresponding pointer and `last`.
4. Copy any remaining `nums2` values.

```
class Solution {
  merge(nums1: number[], m: number, nums2: number[], n: number): void {
    let last = m + n - 1;

    while (m > 0 && n > 0) {
      if (nums1[m - 1] > nums2[n - 1]) {
        nums1[last--] = nums1[m-- - 1];
      } else {
        nums1[last--] = nums2[n-- - 1];
      }
    }

    while (n > 0) {
      nums1[last--] = nums2[n-- - 1];
```

```
    }
  }
}
```

Complexity:

- Time: $O(m+n)$
- Space: $O(1)$ extra space

Approach 3: Three Pointers Without Extra Space (Version II)

Intuition: Cleaner loop: continue only while `j >= 0`, because leftover `nums1` elements are already in correct place.

Algorithm:

1. Set `i = m - 1`, `j = n - 1`, `last = m + n - 1`.
2. While `j >= 0`, place max of `nums1[i]` and `nums2[j]` at `nums1[last]`.
3. Move pointers.

```
class Solution {
  merge(nums1: number[], m: number, nums2: number[], n: number): void {
    let last = m + n - 1;
    let i = m - 1;
    let j = n - 1;

    while (j >= 0) {
      if (i >= 0 && nums1[i] > nums2[j]) {
        nums1[last--] = nums1[i--];
      } else {
        nums1[last--] = nums2[j--];
      }
    }
  }
}
```

Complexity:

- Time: $O(m+n)$
- Space: $O(1)$ extra space

Common Pitfalls

1. Merging from the front in-place This can overwrite values in `nums1` that are still needed.
2. Forgetting leftover `nums2` elements After the main loop, remaining values in `nums2` must be copied.

Your Version (Same In-Place Backward Merge)

Why it works: Your solution uses three pointers from the end of the arrays:

- `writePointer`: where the next largest value should go in `nums1`
- `nums1Pointer`: last valid value in `nums1`
- `nums2Pointer`: last value in `nums2`

By writing from the back, no still-needed values in `nums1` get overwritten.

```
class Solution {
  /**
   * @param {number[]} nums1
   * @param {number} m
   * @param {number[]} nums2
   * @param {number} n
   * @return {void} Do not return anything, modify nums1 in-place instead.
   */
  merge(nums1, m, nums2, n) {
    let writePointer = nums1.length - 1;
    let nums1Pointer = m - 1;
    let nums2Pointer = n - 1;

    while (nums1Pointer >= 0 && nums2Pointer >= 0) {
      if (nums1[nums1Pointer] >= nums2[nums2Pointer]) {
        nums1[writePointer] = nums1[nums1Pointer];
        nums1Pointer--;
      } else {
        nums1[writePointer] = nums2[nums2Pointer];
        nums2Pointer--;
      }
      writePointer--;
    }

    while (nums2Pointer >= 0) {
      nums1[writePointer] = nums2[nums2Pointer];
      writePointer--;
      nums2Pointer--;
    }
  }
}
```

Complexity:

- Time: $O(m + n)$
- Space: $O(1)$ extra space

Quick Sort

Quick sort is another divide-and-conquer sorting algorithm, similar in spirit to merge sort.

Its main idea is to choose a **pivot**, partition the array around that pivot, and then recursively sort the left and right partitions.

Pivot Selection

Common pivot strategies include:

- first element
- last element
- median-of-three
- random pivot

Ideally, the pivot splits the array into two roughly equal halves.

For simplicity, this lesson uses the **last element** as the pivot.

Core Idea

After choosing a pivot, partition the array so that:

- values less than or equal to the pivot go on the left
- values greater than the pivot go on the right

Then recursively quick sort both sides.

Unlike merge sort, there is no merge step afterward. The partitioning itself does the key rearrangement work.

Quick sort can be seen as the opposite of merge sort:

- Merge sort: easy split, more involved merge
- Quick sort: more involved partition, no merge

Randomized quicksort reduces the chance of hitting the worst-case split and gives **expected** $O(n \log n)$ time, although worst-case still remains $O(n^2)$.

Partition Step

The array is rearranged so values smaller than the pivot are on the left side, and the pivot lands in its final sorted position.

This happens in-place, without allocating new arrays for each partition.

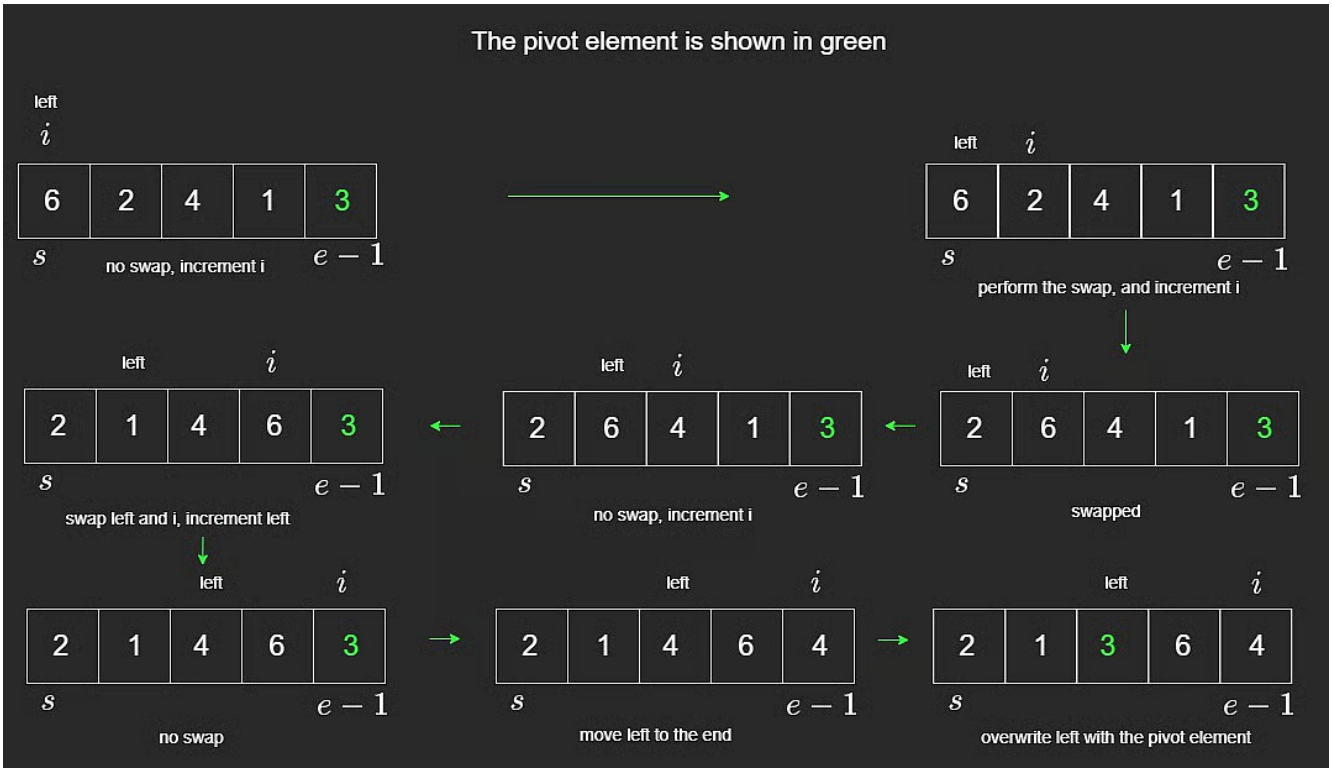


Figure 29: Quick sort partition example

```
function quickSort(arr, s, e) {
  if (e - s + 1 <= 1) {
    return arr;
  }

  const pivot = arr[e];
  let left = s;

  // Partition: elements smaller than pivot on left side
  for (let i = s; i < e; i++) {
    if (arr[i] < pivot) {
      const tmp = arr[left];
      arr[left] = arr[i];
      arr[i] = tmp;
      left++;
    }
  }

  // Move pivot in-between left and right sides
  arr[e] = arr[left];
  arr[left] = pivot;
```

```
  quickSort(arr, s, left - 1);
  quickSort(arr, left + 1, e);

  return arr;
}
```

Time and Space Complexity

Time

- Best case: $O(n \log n)$
- Average case: $O(n \log n)$
- Worst case: $O(n^2)$

Why:

- Partitioning each level costs `O(n)`.
- Best/average case gives about `log n` levels.
- Worst case happens when pivot choices are consistently poor, producing partitions of size `n - 1` and `0`.

Space

- Best/average recursion depth: $O(\log n)$
- Worst-case recursion depth: $O(n)$

Stability

Quick sort is **not stable** because it swaps non-adjacent elements.

Example: `[7,3,7,4,5]`

The two 7s may reverse relative order after partitioning.

NeetCode Practice (Sorting)

1) K Closest Points to Origin

- Difficulty: **Medium**
- Pattern: **Sorting by custom key**
- Core idea: Sort points by squared distance from origin, then return the first `k`.

Problem statement (summary): Given points on a 2D plane and integer `k`, return the `k` closest points to `(0, 0)`.

The Euclidean distance is:

$$\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

But for comparison, we can use squared distance instead:

$$d^2 = x^2 + y^2$$

Example:

- Input: `points = [[0,2],[2,2]], k = 1`
- Output: `[[0,2]]`

Approach 1: Sorting

Intuition: Square root is monotonic, so sorting by squared distance gives the same ordering as sorting by actual distance.

Algorithm:

1. For each point `(x, y)`, compute `x^2 + y^2`.
2. Sort all points by that value.
3. Return the first `k` points.

```
class Solution {
  /**
   * @param {number[][]} points
   * @param {number} k
   * @return {number[][]}
   */
  kClosest(points, k) {
    points.sort((a, b) => a[0] ** 2 + a[1] ** 2 - (b[0] ** 2 + b[1] ** 2));
    return points.slice(0, k);
  }
}
```

Complexity:

- Time: $O(n \log n)$
- Space: depends on sort implementation, often $O(\log n)$ or more

Bucket Sort

Bucket sort is the last sorting algorithm in this section and an important one to know.

It is not as universally used as merge sort or quick sort, but it works very well when the values fall inside a **small known range**.

Core Idea

Suppose we have an array of size `6` containing only values in the range `0` to `2`.

Example:

- input: `[2,1,2,0,0,2]`

We create a bucket for each possible value:

- bucket for `0`
- bucket for `1`
- bucket for `2`

In this lesson, a bucket is just an index in a frequency array.

Once the counts are known, we overwrite the original array in sorted order.

```
function bucketSort(arr) {
  // Assuming arr only contains 0, 1, or 2
  const counts = [0, 0, 0];

  // Count occurrences
  for (let i = 0; i < arr.length; i++) {
    counts[arr[i]] += 1;
  }

  // Rebuild array from bucket counts
  let i = 0;
  for (let n = 0; n < counts.length; n++) {
    for (let j = 0; j < counts[n]; j++) {
      arr[i] = n;
      i++;
    }
  }

  return arr;
}
```

How the Pointers Work

In the second phase:

- `i` = next write position in original array
- `n` = current bucket/value being written
- `j` = how many times current value has been placed so far

Example:

- `counts = [2,1,3]`

- original `arr = [2,1,2,0,0,2]`

When `n = 0`, we write `0` two times. When `n = 1`, we write `1` one time. When `n = 2`, we write `2` three times.

Final result:

- `[0,0,1,2,2,2]`

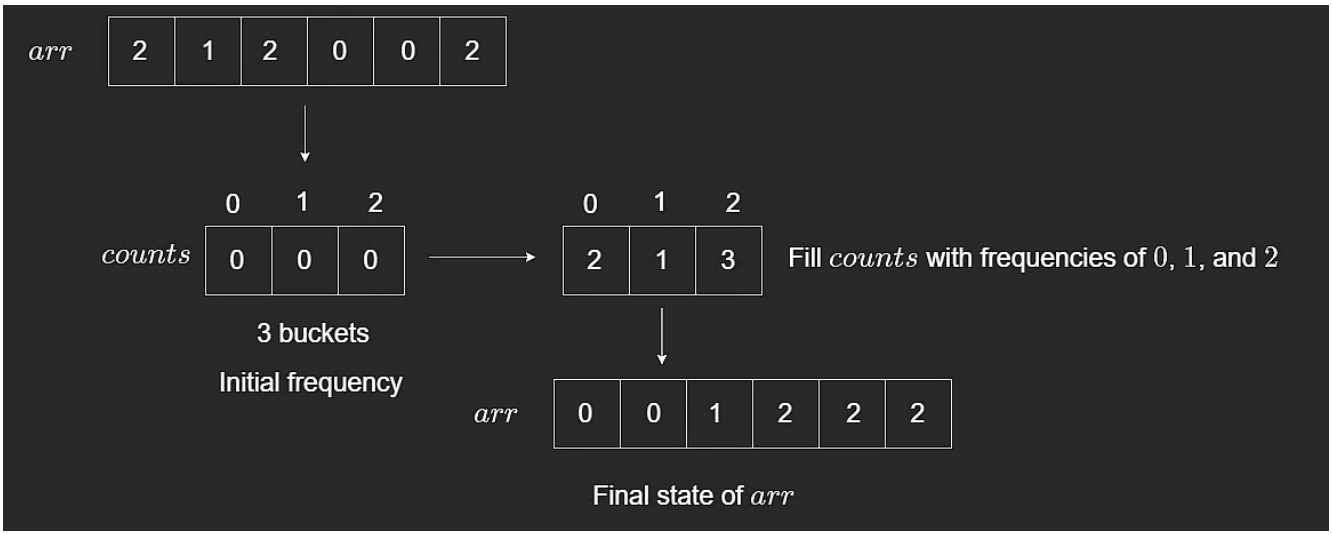


Figure 30: Bucket sort frequency rewrite

Time Complexity

At first glance, the nested loops may look like $O(n^2)$, but that is not the right analysis.

Why:

- First loop counts all elements: `O(n)`
- Second phase writes each original element exactly once total

So total time is linear in the size of the input plus the bucket range.

For this fixed small-range example, we treat it as:

- Time: $O(n)$

This is a good reminder that nested loops do not always mean quadratic time.

Stability

This implementation is **not stable**.

Because the array is overwritten from frequency counts rather than preserving original element order, relative order among equal items is not kept.

Closing Notes

Bucket sort is powerful when the dataset lies in a specific bounded range.

For general-purpose algorithm problems, merge sort and quick sort are usually safer defaults.

NeetCode Practice (Sorting)

1) Sort Colors

- Difficulty: **Medium**
- Pattern: **Counting sort / bucket frequency**
- Core idea: Count how many `0`s, `1`s, and `2`s appear, then overwrite the array in color order.

Problem statement (summary): You are given an array `nums` where:

- `0` = red
- `1` = white
- `2` = blue

Sort the array in-place so colors are grouped as `0, 1, 2`.

Built-in sort functions are not allowed.

Example:

- Input: `nums = [1,0,1,2]`
- Output: `[0,1,1,2]`

Approach 1: Counting Sort

Intuition: There are only three distinct values, so it is efficient to count each one and then reconstruct the array.

Algorithm:

1. Count occurrences of `0`, `1`, and `2`.
2. Overwrite array with all `0`s first.
3. Then write all `1`s.
4. Then write all `2`s.

```
class Solution {
  /**
   * @param {number[]} nums
   * @return {void} Do not return anything, modify nums in-place instead.
   */
  sortColors(nums) {
    const bucket = new Array(3).fill(0);

    for (let i = 0; i < nums.length; i++) {
      const num = nums[i];
      bucket[num]++;
    }

    let j = 0;

    for (let i = 0; i < bucket.length; i++) {
      while (bucket[i] > 0) {
        nums[j] = i;
        j++;
        bucket[i]--;
      }
    }
  }
}
```

Complexity:

- Time: $O(n)$
- Space: $O(1)$ since bucket size is fixed at 3

Sorting Time Complexity Cheat Sheet

Use this chart as a quick reference for comparing the sorting algorithms covered in this module.

It is especially useful when deciding between:

- simpler algorithms like insertion sort
- divide-and-conquer algorithms like merge sort and quick sort
- range-based algorithms like bucket sort

Visual Summary

Algorithm	Big-O Time Complexity	Note
Insertion Sort	$O(n^2)^*$	If fully, or nearly sorted, $O(n)$
Merge Sort	$O(n\ log\ n)$	
Quick Sort	$O(n\ log\ n)^*$	In worst case it is $O(n^2)$
Bucket Sort	$O(n)^*$	Assuming all values in the input are in a specified range.

Figure 31: Sorting time complexity cheat sheet

How to Use It

When choosing a sorting approach, ask:

1. Is the array small or nearly sorted?
2. Do I need guaranteed $O(n \log n)$ performance?
3. Is the value range small enough for counting or bucket-style sorting?
4. Do I care whether the algorithm is stable?

5. Binary Search

Binary Search (Search an Array)

Binary search is an efficient way to find a target in a **sorted array**.

It works by repeatedly checking the middle element and discarding half of the search space each time.

Think of searching for a word in a dictionary: open near the middle, decide left or right, and repeat.

Core Idea

At each step:

- compare `mid` with the target
- if target is larger, search the right half
- if target is smaller, search the left half
- if equal, return the index

Two common binary-search problem types:

- Search Array: find a target in a sorted array
- Search Range: search over a numeric range using a helper or predicate

Example Walkthrough

Array: `[1,2,3,4,5,6,7,8]`

Target: `5`

Step-by-step:

1. Start with the full array.
2. Check middle index.
3. Since `5 > 4`, move right.
4. Check the new middle.
5. Since `5 < 6`, move left.
6. The target is found.

Implementation

Use two pointers:

- `L` / `left` / `low`
- `R` / `right` / `high`

Midpoint formula:

$$mid = L + \frac{R - L}{2}$$

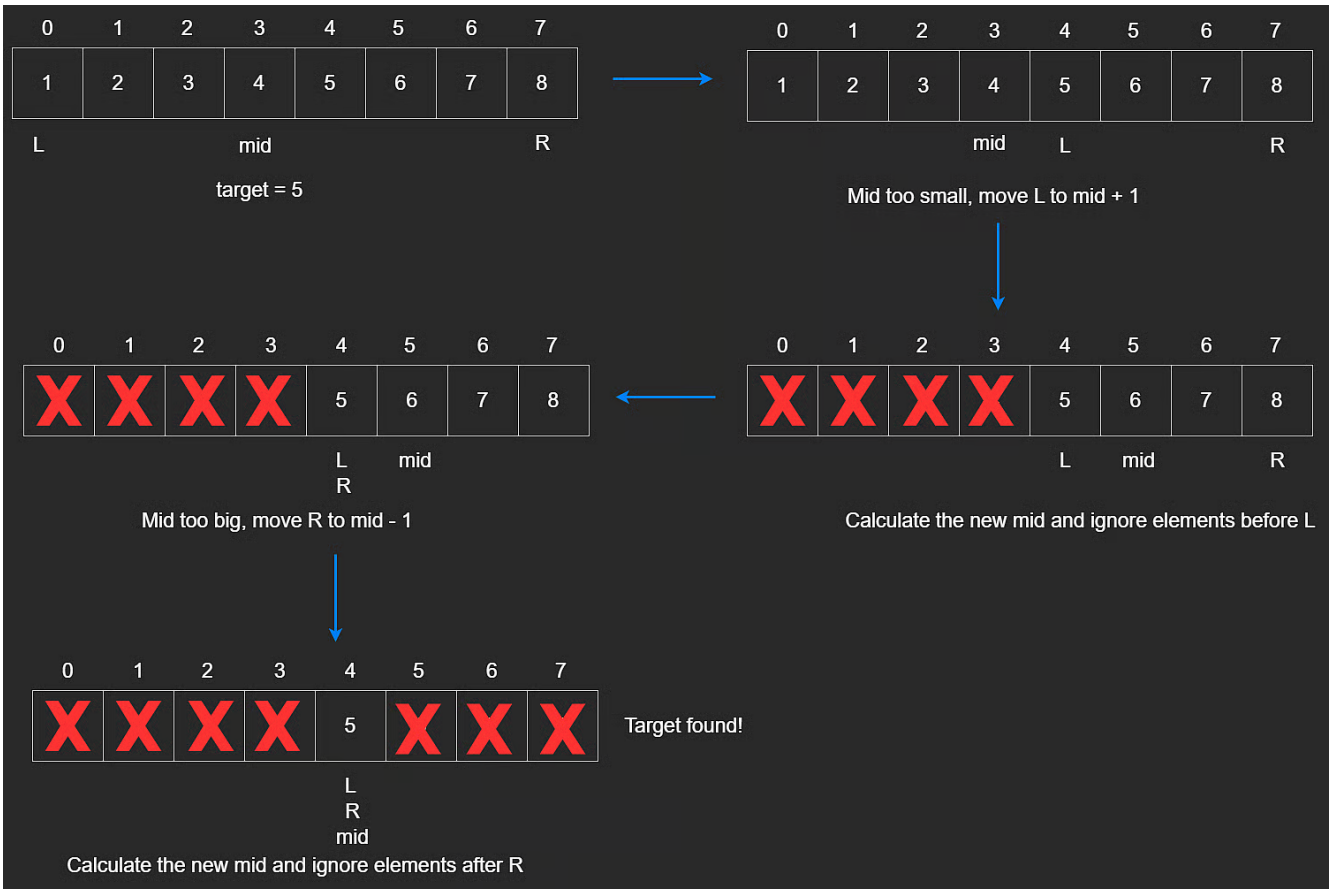


Figure 32: Binary search on array

This is preferred over `(L + R) / 2` because it avoids overflow in languages with fixed-size integers.

```
const arr = [1, 3, 3, 4, 5, 6, 7, 8];

function binarySearch(arr, target) {
  let L = 0;
  let R = arr.length - 1;

  while (L <= R) {
    let mid = Math.floor((L + R) / 2);

    if (target > arr[mid]) {
      L = mid + 1;
    } else if (target < arr[mid]) {
      R = mid - 1;
    } else {
      return mid;
    }
  }

  return -1;
}
```

Target Not Found

If the target does not exist, the search space eventually becomes empty.

At that point, `L > R`.

Time and Space Complexity

Binary search halves the search space each time.

If `n` is the array size, then the number of halvings is about:

$$1 = \frac{n}{2^x}$$

Solving gives:

$$x = \log_2(n)$$

So:

- Time: $O(\log n)$
- Space: $O(1)$

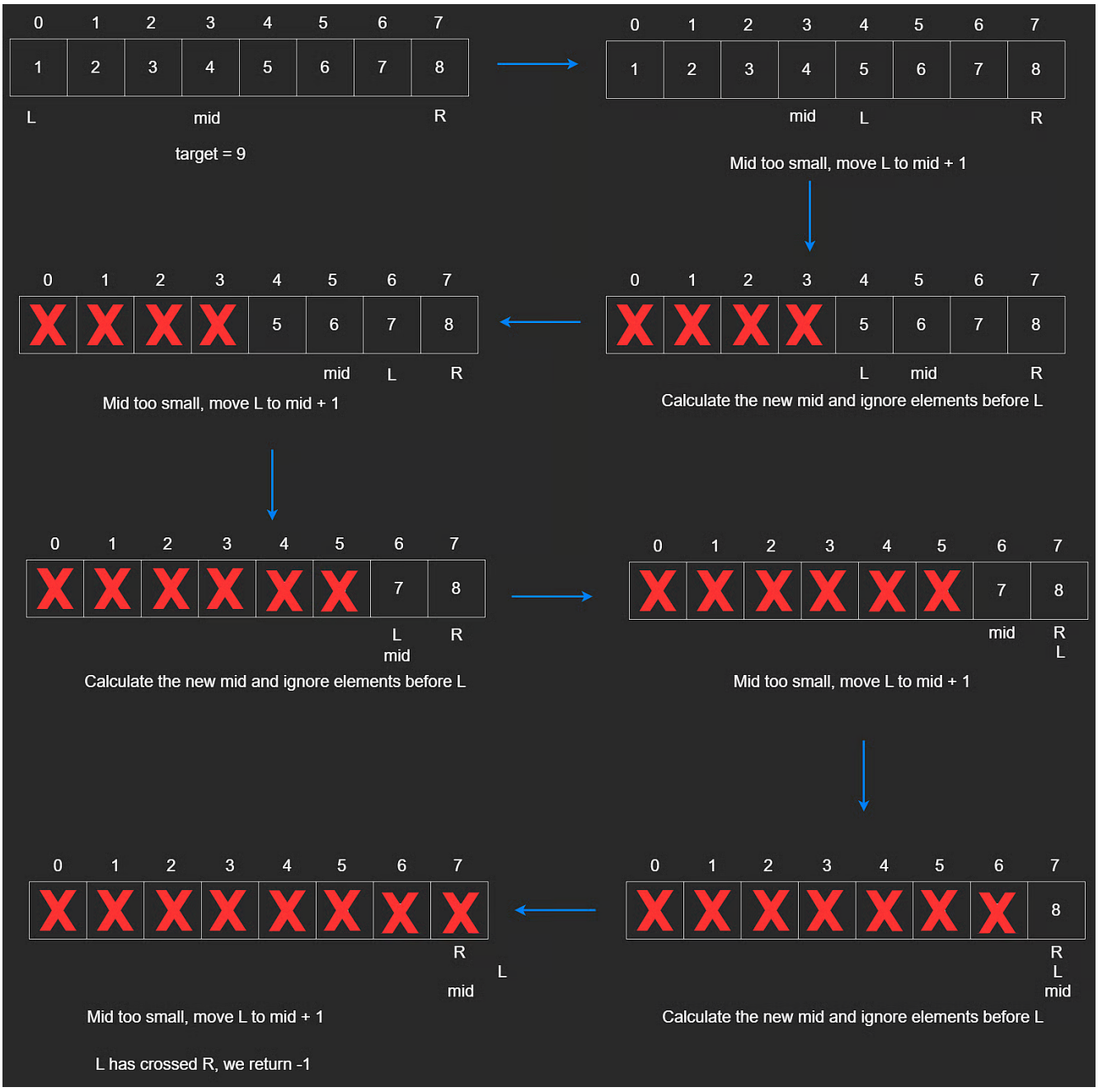


Figure 33: Target not found in sorted array

NeetCode Practice (Binary Search)

1) Binary Search

- Difficulty: **Easy**
- Pattern: **Iterative binary search**
- Core idea: Shrink the search range until the target is found or the pointers cross.

Problem statement (summary): Given a sorted array of distinct integers `nums` and a target, return the index of `target` or `-1`.

Example:

- Input: `nums = [-1,0,2,4,6,8], target = 4`
- Output: `3`

Algorithm:

1. Set `l = 0` and `r = nums.length - 1`.
2. While `l <= r`:
3. Compute middle `m`.
4. If `nums[m] == target`, return `m`.
5. If `nums[m] < target`, search right half.
6. If `nums[m] > target`, search left half.
7. If not found, return `-1`.

```
class Solution {
  search(nums: number[], target: number): number {
    let l = 0;
    let r = nums.length - 1;

    while (l <= r) {
      const m = l + Math.floor((r - l) / 2);

      if (nums[m] > target) {
        r = m - 1;
      } else if (nums[m] < target) {
        l = m + 1;
      } else {
        return m;
      }
    }

    return -1;
  }
}
```

Complexity:

- Time: $O(\log n)$
- Space: $O(1)$

2) Search Range

- Difficulty: **Medium**
- Pattern: **Lower bound binary search**
- Core idea: Find the first index where a value is greater than or equal to the target.

Problem statement (summary): Find the first occurrence of a target in a sorted array, or return `-1` if it does not exist.

Intuition: Instead of searching for equality directly, search for the leftmost index where the target could appear.

This naturally handles duplicates and avoids overshooting.

Algorithm:

1. Set `l = 0`, `r = nums.length`.
2. While `l < r`:
3. Compute `m`.
4. If `nums[m] >= target`, move left: `r = m`.
5. Otherwise move right: `l = m + 1`.
6. After the loop, check whether `nums[l] === target`.
7. Return the index or `-1`.

```
class Solution {
  search(nums: number[], target: number): number {
    let l = 0;
    let r = nums.length;

    while (l < r) {
      const m = l + Math.floor((r - l) / 2);

      if (nums[m] >= target) {
        r = m;
      } else {
        l = m + 1;
      }
    }

    return l < nums.length && nums[l] === target ? l : -1;
  }
}
```

Complexity:

- Time: $O(\log n)$
- Space: $O(1)$

3) Search a 2D Matrix

- Difficulty: **Medium**
- Pattern: **Binary search on flattened indices**
- Core idea: Treat the matrix like one sorted array and map indices back to row/column coordinates.

Problem statement (summary): Given an `m x n` matrix where each row is sorted and the first value of each row is greater than the last value of the previous row, determine whether `target` exists.

Example:

- Input: `matrix = [[1,2,4,8],[10,11,12,13],[14,20,30,40]], target = 10`
- Output: `true`

Intuition: Because the matrix is globally sorted row by row, it behaves like one big sorted array.

Index mapping:

- `row = mid / columns`
- `col = mid % columns`

Algorithm:

1. Set `left = 0`, `right = rows * columns - 1`.
2. While `left <= right`:
3. Compute `mid`.
4. Convert `mid` to `row` and `col`.
5. Compare matrix value to target.
6. Move left or right accordingly.
7. Return `false` if no match is found.

```
class Solution {
  searchMatrix(matrix: number[][], target: number): boolean {
    const rows = matrix.length;
    const columns = matrix[0].length;

    let left = 0;
    let right = rows * columns - 1;

    while (left <= right) {
      const mid = Math.floor((left + right) / 2);
      const col = mid % columns;
      const row = Math.floor(mid / columns);

      if (matrix[row][col] > target) {
        right = mid - 1;
```

```
      } else if (matrix[row][col] < target) {
        left = mid + 1;
      } else {
        return true;
      }
    }

    return false;
  }
}
```

Complexity:

- Time: $O(\log(mn))$
- Space: $O(1)$

Binary Search (Search Range)

Search range binary search applies the same core idea as array search, but the search space is a **numeric range** rather than an array.

Instead of comparing against `arr[mid]`, we compare a guessed value against a hidden answer through a helper function.

Concept

Imagine guessing a number from `1` to `100`.

After each guess, you are told whether it is:

- correct
- too small
- too large

That feedback is enough to shrink the search space just like normal binary search.

In many problems, the comparison is hidden behind an API or function such as `isCorrect(n)`.

```
// Return 1 if n is too big, -1 if too small, 0 if correct
function isCorrect(n) {
  if (n > 10) {
    return 1;
  } else if (n < 10) {
    return -1;
  } else {
    return 0;
  }
}
```

Range Binary Search

```
function binarySearch(low, high) {
  while (low <= high) {
    const mid = Math.floor((low + high) / 2);

    if (isCorrect(mid) > 0) {
      high = mid - 1;
    } else if (isCorrect(mid) < 0) {
      low = mid + 1;
    } else {
      return mid;
    }
  }

  return -1;
}
```

This style is common in problems where the judge function is a black box.

Time and Space Complexity

- Time: $O(\log n)$
- Space: $O(1)$

NeetCode Practice (Binary Search)

1) Guess Number Higher or Lower

- Difficulty: **Easy**
- Pattern: **Binary search on a range**
- Core idea: Use the `guess()` API to determine whether the hidden number is higher or lower.

Problem statement (summary): Pick a number from `1` to `n`. Each wrong guess returns whether the target is higher or lower. Return the picked number.

Example:

- Input: `n = 5, pick = 3`
- Output: `3`

Algorithm:

1. Set `low = 1`, `high = n`.
2. While `low <= high`:
3. Compute `mid`.
4. Call `guess(mid)`.

5. If result is `0`, return `mid`.
6. If result is `1`, search right half.
7. If result is `-1`, search left half.

```
/**
 * Forward declaration of guess API.
 * @param {number} num your guess
 * @return -1 if num is higher than the picked number
 *          1 if num is lower than the picked number
 *          otherwise return 0
 * function guess(num) {}
 */

class Solution {
  guessNumber(n: number): number {
    let low = 1;
    let high = n;

    while (low <= high) {
      const mid = Math.floor((low + high) / 2);
      const guessResult = guess(mid);

      if (guessResult === 1) {
        low = mid + 1;
      } else if (guessResult === -1) {
        high = mid - 1;
      } else {
        return mid;
      }
    }

    return -1;
  }
}
```

Complexity:

- Time: $O(\log n)$
- Space: $O(1)$

2) Search Range

- Difficulty: **Medium**
- Pattern: **Lower bound binary search**
- Core idea: Find the first index where the target could appear, then verify it.

Problem statement (summary): Given a sorted array, return the first occurrence of a target or `-1`.

Intuition: Binary search for the leftmost position avoids overshooting and handles duplicates naturally.

Algorithm:

1. Set `l = 0`, `r = nums.length`.
2. While `l < r`:
3. Compute `m`.
4. If `nums[m] >= target`, move `r = m`.
5. Otherwise set `l = m + 1`.
6. After the loop, verify `nums[l] === target`.

```
class Solution {
  search(nums: number[], target: number): number {
    let l = 0;
    let r = nums.length;

    while (l < r) {
      const m = l + Math.floor((r - l) / 2);

      if (nums[m] >= target) {
        r = m;
      } else {
        l = m + 1;
      }
    }

    return l < nums.length && nums[l] === target ? l : -1;
  }
}
```

Complexity:

- Time: $O(\log n)$
- Space: $O(1)$

3) Koko Eating Bananas

- Difficulty: **Medium**
- Pattern: **Binary search on answer space**
- Core idea: The faster Koko eats, the fewer hours she needs. This monotonic behavior allows binary search over possible speeds.

Problem statement (summary): Given `piles`, where `piles[i]` is the number of bananas in the i-th pile, and a limit `h` hours, find the minimum integer eating speed `k` such that all bananas are eaten within `h` hours.

Each hour, Koko chooses one pile and eats up to `k` bananas from it. If the pile has fewer than `k`, she finishes it and cannot continue with another pile in the same hour.

Examples:

- Input: `piles = [1,4,3,2], h = 9`
- Output: `2`
- Input: `piles = [25,10,23,4], h = 4`
- Output: `25`

Approach 1: Binary Search

Intuition: Trying every speed from `1` to `max(piles)` is too slow. But the total required time decreases as speed increases, which gives us a sorted/monotonic search space.

So we binary-search the speed:

- low speed -> too many hours
- high speed -> feasible hours

Then we find the smallest feasible speed.

Algorithm:

1. Set `left = 1`, `right = max(piles)`.
2. Track best answer `res`.
3. While `left <= right`:
4. Test speed `mid`.
5. Compute hours needed: `sum(ceil(pile / mid))`.
6. If hours `<= h`, record `mid` and search left half.
7. Otherwise search right half.
8. Return `res`.

```javascript
class Solution {
  /**
   * @param {number[]} piles
   * @param {number} h
   * @return {number}
   */
  minEatingSpeed(piles, h) {
    let left = 1;
    let right = Math.max(...piles);
    let res = right;

    while (left <= right) {
      const mid = Math.floor((left + right) / 2);

      let totalTime = 0;
      for (const pile of piles) {
        totalTime += Math.ceil(pile / mid);
      }

      if (totalTime <= h) {
        res = mid;
        right = mid - 1;
      } else {
        left = mid + 1;
      }
    }

    return res;
  }
}
```

Complexity:

- Time: $O(n \log m)$
- Space: $O(1)$

Where:

- `n` is the number of piles
- `m` is `max(piles)`

Binary Trees

Concept

Similar to linked lists, binary trees are data structures built from nodes and pointers.

In a linked list, nodes form a straight line with `next` (and sometimes `prev`).

In a binary tree, each node has at most two child pointers:

- `left`
- `right`

The first node is called the **root**.

```
class TreeNode {
  constructor(val) {
    this.val = val;
    this.left = null;
    this.right = null;
  }
}
```

Node categories:

- **Leaf node**: no children
- **Non-leaf node**: at least one child

Unlike linked lists, tree edges conceptually point downward from parent to child, and cycles are not allowed.

Tree Terminology

Root Node

The root is the top-most node and has no parent. Every node in the tree is reachable from the root.

Leaf Nodes

Leaf nodes have no children. The bottom-most level is all leaves, but leaves can also appear earlier in uneven trees.

Height

Tree height is measured from the root down to the lowest leaf.

Two common conventions:

- count nodes in the path
- count edges in the path

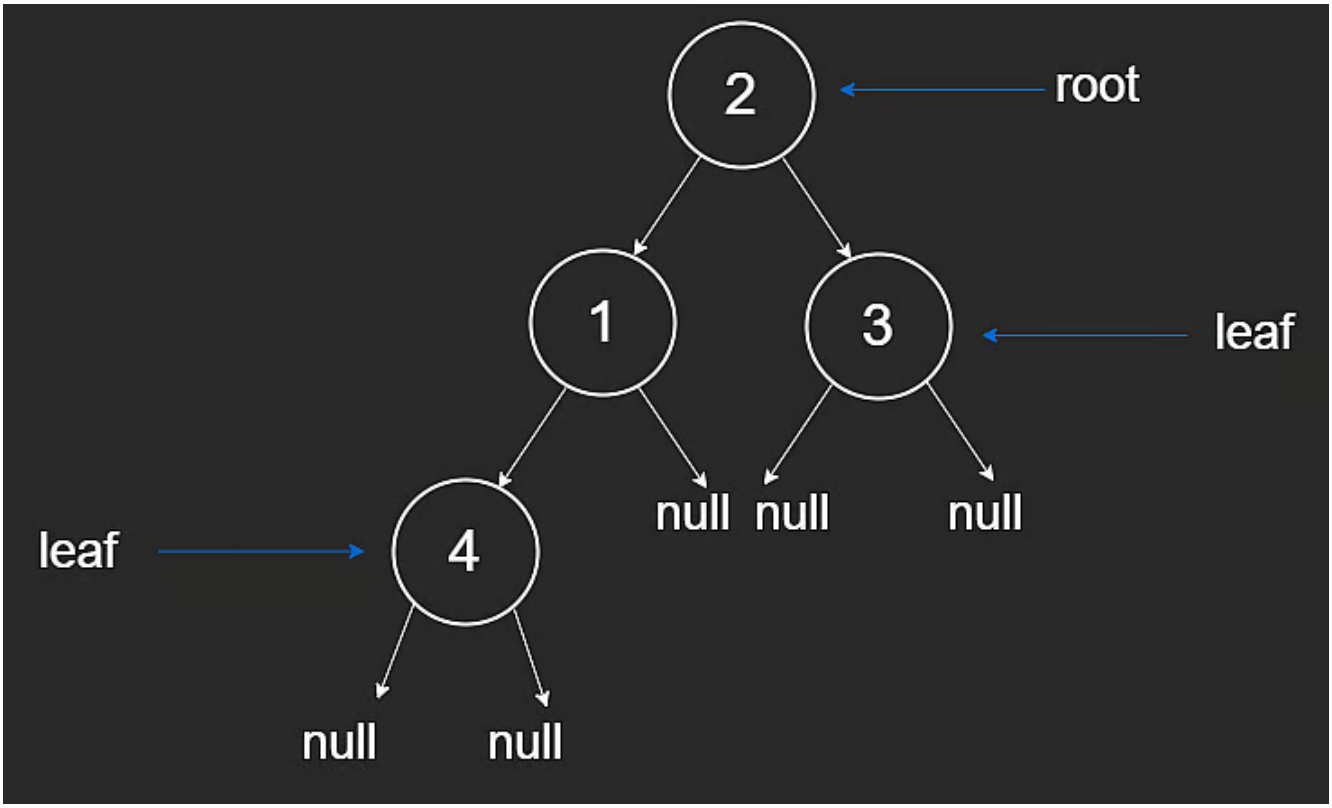


Figure 34: Binary tree representation

For a path with `n` nodes, edge count is `n - 1`.

Example from the diagram:

- height by nodes: `3`
- height by edges: `2`

General fact for trees:

- number of edges = `n - 1` where `n` is number of nodes

Depth

Depth of a node is measured from that node upward to the root.

As you move down the tree, depth increases.

Like height, depth may be measured by nodes or edges depending on convention.

Ancestor and Descendant

Ancestor

A node connected to nodes below it in the tree is an ancestor of those nodes.

The root is an ancestor of every node.

Descendant

A descendant of a node is:

- its child, or
- child of one of its descendants recursively

Binary Search Trees

Binary Trees vs Binary Search Trees

A **Binary Search Tree (BST)** is a binary tree with an added sorted property:

- every value in the left subtree is smaller than the current node
- every value in the right subtree is greater than the current node

This property is recursive and applies at every node.

It is conceptually similar to a sorted array, but with tree structure.

Why BSTs

BSTs are useful because they can support:

- search in $O(\log n)$ on balanced trees
- insertion in $O(\log n)$ on balanced trees
- deletion in $O(\log n)$ on balanced trees

Sorted arrays can search in $O(\log n)$, but insertion and deletion are typically $O(n)$ due to shifting.

BST Search

Because trees are recursive structures, recursion is a natural traversal method.

Example tree: `[2,1,3,null,null,null,4]`

Target: `3`

Search logic:

- if target is greater than current value, go right
- if target is smaller, go left
- if equal, return `true`

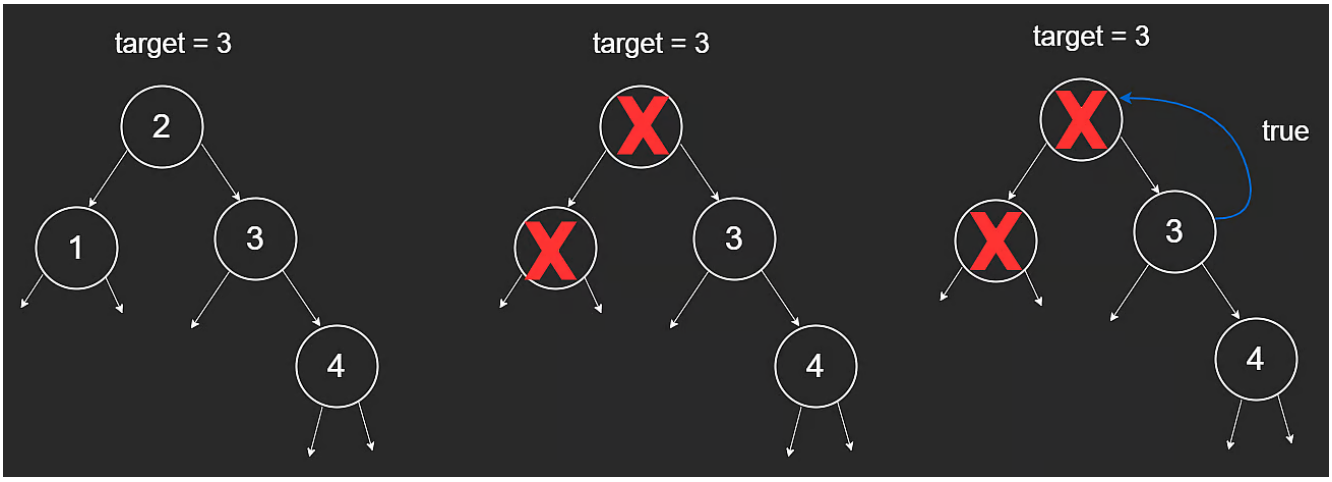


Figure 35: BST search path

```
function search(root, target) {
  if (root == null) {
    return false;
  }

  if (target > root.val) {
    return search(root.right, target);
  } else if (target < root.val) {
    return search(root.left, target);
  } else {
    return true;
  }
}
```

How It Works

1. If node is `null`, target does not exist in that subtree.
2. If target is greater, recurse right.
3. If target is smaller, recurse left.
4. If equal, return `true`.

Base cases:

- `root == null -> false`
- `root.val == target -> true`

Time Complexity

Let `h` be tree height.

- Search cost is proportional to height: $O(h)$

So:

- Balanced BST: $O(\log n)$
- Skewed BST (worst case): $O(n)$

A balanced tree means left and right subtree heights are close (difference at most 1).

Insertion and Removal in a Binary Search Tree

One major advantage of BSTs over sorted arrays is efficient insertion and deletion.

On a balanced tree, these operations are typically $O(\log n)$.

Insertion

To insert a value:

1. Traverse the tree to find where it should go.
2. Add it as a leaf while preserving BST order.

Example from root `[4]`, inserting `6`:

- `[4,null,6]`
- `[6,4,null]`

Both satisfy BST rules, but adding as a leaf is the standard process.

If we start with `[4,null,6]` and insert `5`, we get `[4,null,6,5,null]`.

```
// Insert a new node and return the root of the BST.
function insert(root, val) {
  if (root == null) {
    return new TreeNode(val);
  }

  if (val > root.val) {
    root.right = insert(root.right, val);
  } else if (val < root.val) {
    root.left = insert(root.left, val);
  }

  return root;
}
```

Why this works:

- null spot means insertion point
- recursive return values reconnect to `root.left` or `root.right`
- tree structure remains valid

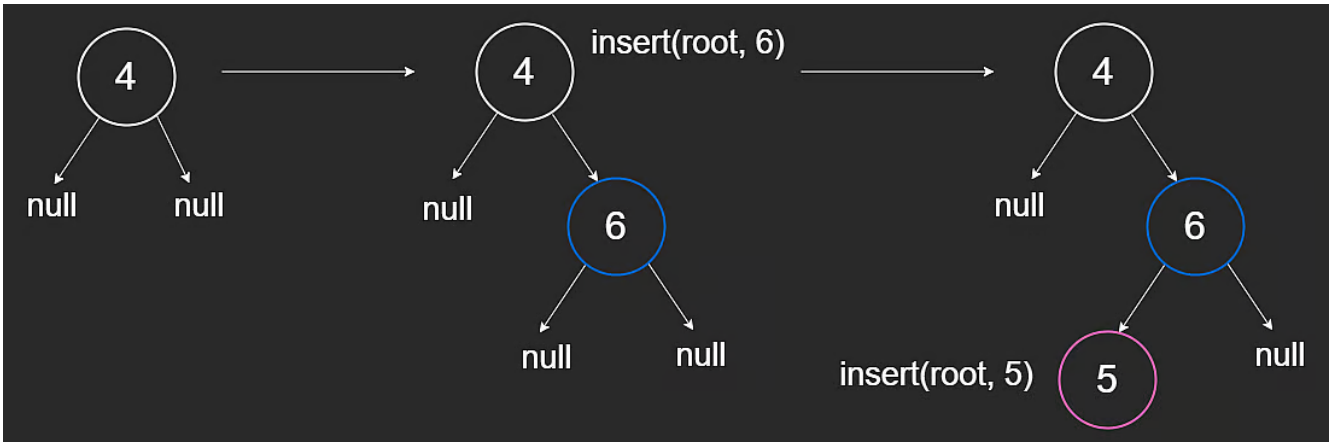


Figure 36: BST insertion example

Removal

Deletion depends on number of children in target node.

Case 1: 0 or 1 Child

- No child: remove node directly.
- One child: connect parent to that child.

Examples:

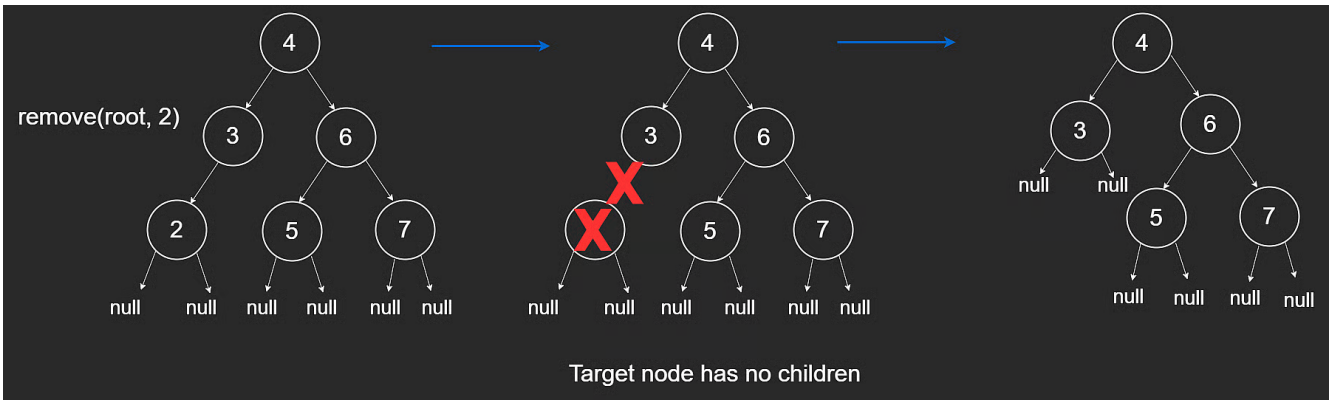


Figure 37: BST removal no child

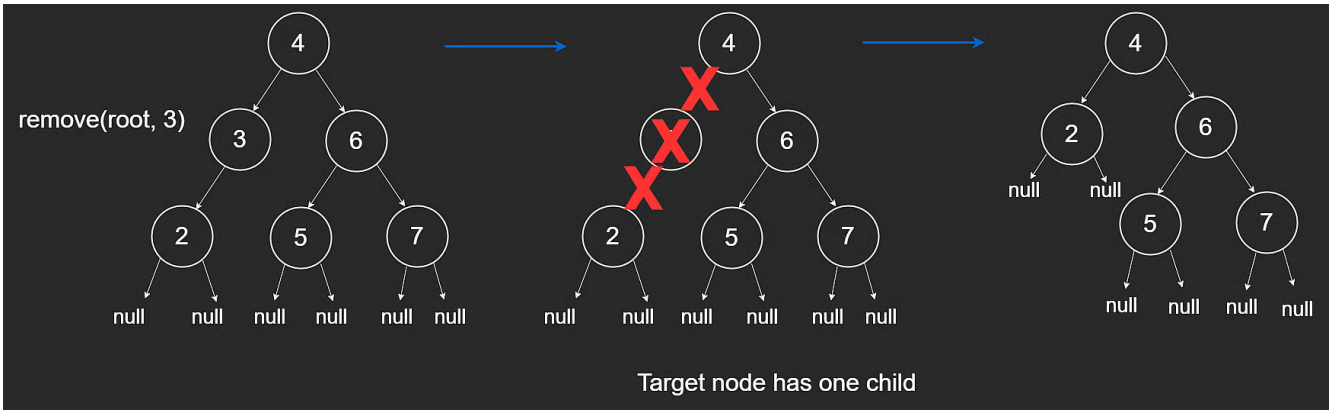


Figure 38: BST removal one child

Case 2: 2 Children

Replace target with its **in-order successor**:

- the left-most node in target's right subtree
- equivalently, the smallest value greater than target

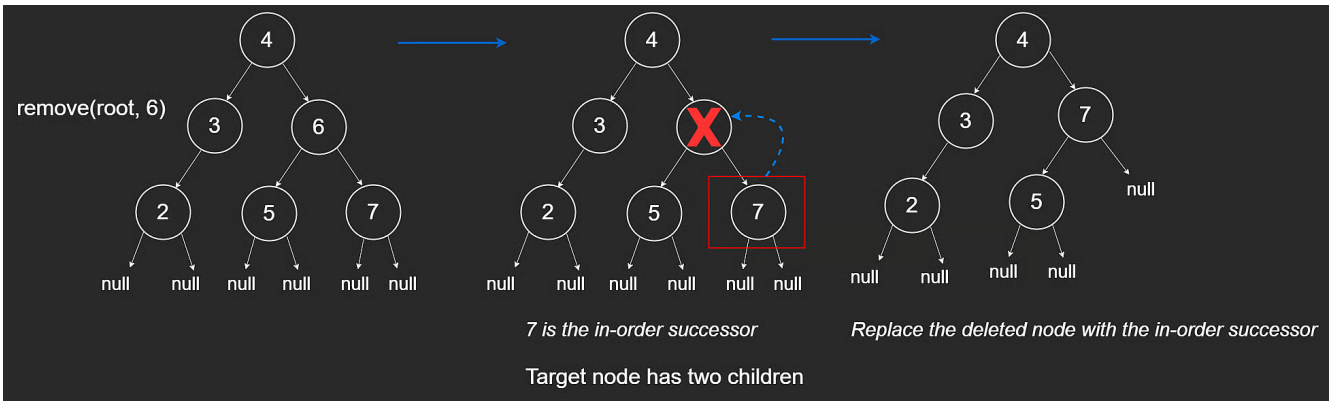


Figure 39: BST removal two children

```
// Return the minimum value node of the BST.
function minValueNode(root) {
  let curr = root;
  while (curr != null && curr.left != null) {
    curr = curr.left;
  }
  return curr;
}

// Remove a node and return the root of the BST.
function remove(root, val) {
  if (root == null) {
    return null;
```

```
  }

  if (val > root.val) {
    root.right = remove(root.right, val);
  } else if (val < root.val) {
    root.left = remove(root.left, val);
  } else {
    if (root.left == null) {
      return root.right;
    } else if (root.right == null) {
      return root.left;
    } else {
      const minNode = minValueNode(root.right);
      root.val = minNode.val;
      root.right = remove(root.right, minNode.val);
    }
  }

  return root;
}
```

Time and Space Complexity

Let `h` be tree height.

- Insertion time: $O(h)$
- Removal time: $O(h)$

So:

- Balanced BST: $O(\log n)$
- Worst-case skewed BST: $O(n)$

Recursive space:

- Balanced: $O(\log n)$
- Worst-case: $O(n)$

NeetCode Practice (BST)

1) Insert into a Binary Search Tree

- Difficulty: **Medium**
- Pattern: **BST insertion (recursion and iteration)**
- Core idea: Follow BST ordering until you hit a null child, then place the new node there.

Problem statement (summary): Given a BST root and a value `val`, insert `val` into the BST and return the root.

Multiple valid resulting trees may exist as long as BST property holds.

Example:

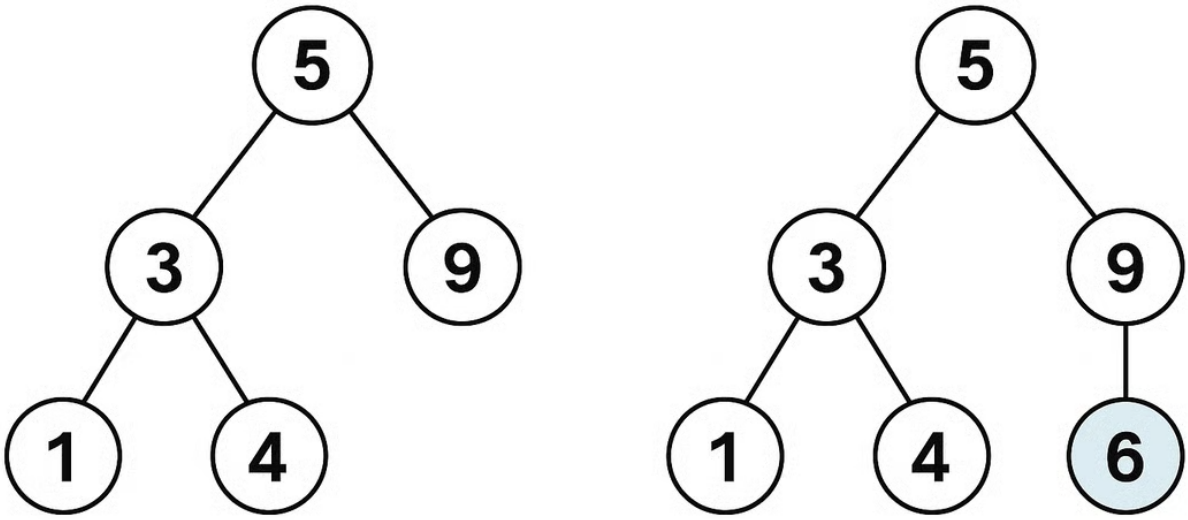


Figure 40: Insert example

- Input: `root = [5,3,9,1,4], val = 6`
- Output: `[5,3,9,1,4,6]`

Approach 1: Recursion Intuition: Use BST comparisons to recurse left or right. Null means insertion point.

Algorithm:

1. If root is null, return new node.
2. If `val > root.val`, recurse right.

3. Else recurse left.
4. Return root.

```
/**
 * Definition for a binary tree node.
 * class TreeNode {
 *     constructor(val = 0, left = null, right = null) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
  insertIntoBST(root: TreeNode | null, val: number): TreeNode {
    if (!root) {
      return new TreeNode(val);
    }

    if (val > root.val) {
      root.right = this.insertIntoBST(root.right, val);
    } else {
      root.left = this.insertIntoBST(root.left, val);
    }

    return root;
  }
}
```

Complexity:

- Time: $O(h)$
- Space: $O(h)$ recursion stack

Approach 2: Iteration **Intuition:** Same decision process as recursion, but with a loop.

Algorithm:

1. If root is null, return new node.
2. Traverse with `curr`.
3. Move left/right by comparison.
4. Insert once the desired child pointer is null.

```typescript
/**
 * Definition for a binary tree node.
 * class TreeNode {
 *     constructor(val = 0, left = null, right = null) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
  insertIntoBST(root: TreeNode | null, val: number): TreeNode {
    if (!root) {
      return new TreeNode(val);
    }

    let curr: TreeNode = root;

    while (true) {
      if (curr.val < val) {
        if (!curr.right) {
          curr.right = new TreeNode(val);
          return root;
        }
        curr = curr.right;
      } else {
        if (!curr.left) {
          curr.left = new TreeNode(val);
          return root;
        }
        curr = curr.left;
      }
    }
  }
}
```

Complexity:

- Time: $O(h)$
- Space: $O(1)$ extra space

2) Delete Node in a BST

- Difficulty: **Medium**
- Pattern: **BST deletion with structural cases**
- Core idea: Find node, then handle 0-child, 1-child, or 2-child cases.

Problem statement (summary): Given BST root and key, delete node with `key` if present and return updated root.

Example:

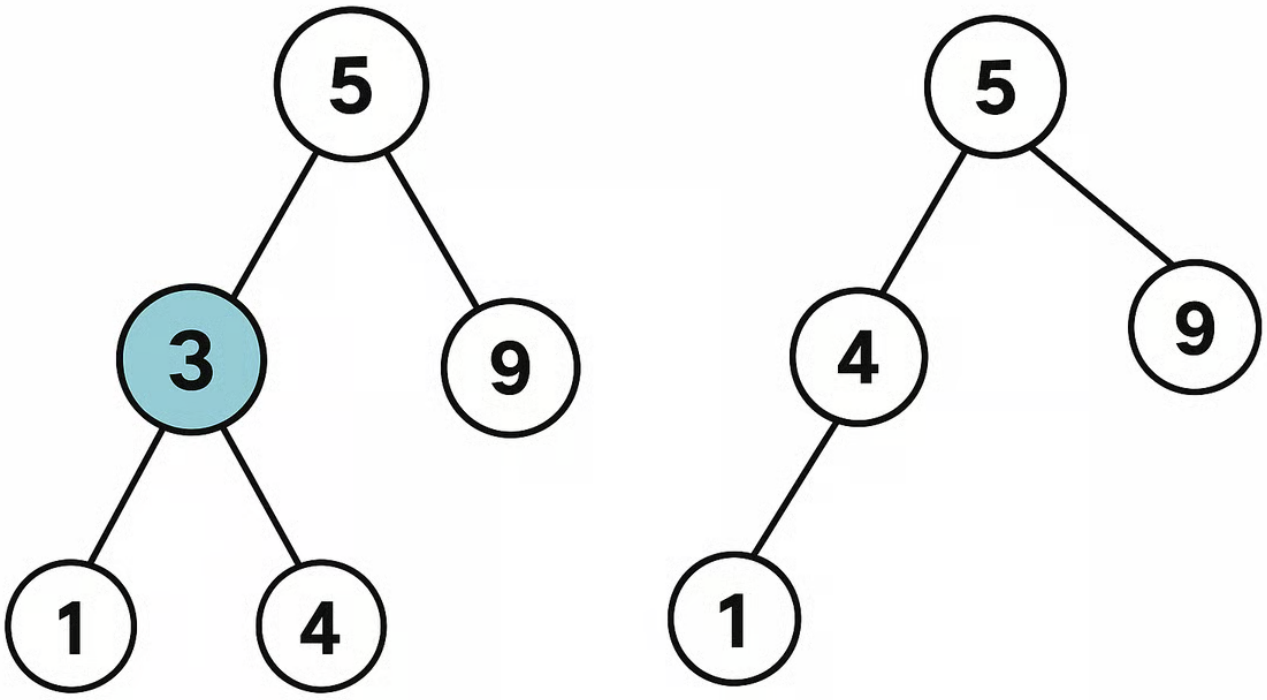


Figure 41: Delete example

- Input: `root = [5,3,9,1,4], key = 3`
- Output: `[5,4,9,1]`

Approach 1: Recursion **Intuition:** Recursively search for target. For two-child case, use left-most node of right subtree as replacement anchor.

Algorithm:

1. Recurse to find key.
2. If found and one side is null, return the other side.
3. If both sides exist, find successor in right subtree.
4. Attach deleted node's left subtree to successor's left.

5. Return right subtree as replacement root.

```
class Solution {
  deleteNode(root: TreeNode | null, key: number): TreeNode | null {
    if (!root) return root;

    if (root.val > key) {
      root.left = this.deleteNode(root.left, key);
    } else if (root.val < key) {
      root.right = this.deleteNode(root.right, key);
    } else {
      if (!root.right) {
        return root.left;
      }

      if (!root.left) {
        return root.right;
      }

      let curr = root.right;
      while (curr.left) {
        curr = curr.left;
      }

      curr.left = root.left;
      return root.right;
    }

    return root;
  }
}
```

Complexity:

- Time: $O(h)$
- Space: $O(h)$ recursion stack

Approach 2: Iteration **Intuition:** Find node and parent with a loop, then update pointers directly.

Algorithm:

1. Locate node and parent.
2. If 0 or 1 child, replace node with child.
3. If 2 children, locate inorder successor and its parent.
4. Rewire successor and attach to deleted node position.

```typescript
/**
 * Definition for a binary tree node.
 * class TreeNode {
 *     constructor(val = 0, left = null, right = null) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */
class Solution {
  deleteNode(root: TreeNode | null, key: number): TreeNode | null {
    if (!root) return root;

    let parent: TreeNode | null = null;
    let cur: TreeNode | null = root;

    while (cur && cur.val !== key) {
      parent = cur;
      if (key > cur.val) {
        cur = cur.right;
      } else {
        cur = cur.left;
      }
    }

    if (!cur) return root;

    if (!cur.left || !cur.right) {
      const child = cur.left || cur.right;
      if (!parent) return child;
      if (parent.left === cur) {
        parent.left = child;
      } else {
        parent.right = child;
      }
    } else {
      let par: TreeNode | null = null;
      const delNode = cur;
      cur = cur.right;

      while (cur.left) {
        par = cur;
```

```
        cur = cur.left;
      }

      if (par) {
        par.left = cur.right;
        cur.right = delNode.right;
      }
      cur.left = delNode.left;

      if (!parent) return cur;

      if (parent.left === delNode) {
        parent.left = cur;
      } else {
        parent.right = cur;
      }
    }

    return root;
  }
}
```

Complexity:

- Time: $O(h)$
- Space: $O(1)$ extra space

Where `h` is the tree height for both approaches.

Depth-First Search

Depth-First Search (DFS) is one of the most common interview algorithms.

It is used for both trees and graphs. This lesson focuses on trees.

Concept

To traverse an entire tree, we must visit every node.

DFS goes as deep as possible along one path before backtracking.

For example:

1. Keep moving left until `null`.
2. Backtrack to parent.
3. Explore right subtree.

DFS is naturally implemented with recursion, but can also be implemented iteratively with a stack.

DFS Traversal Orders

There are three standard traversal orders:

- Inorder
- Preorder
- Postorder

Inorder Traversal

Order: **left -> node -> right**

```
function inorder(root) {
  if (root == null) {
    return;
  }
  inorder(root.left);
  console.log(root.val);
  inorder(root.right);
}
```

Example order in diagram: `[2,3,4,5,6,7]`

Important:

- Inorder is sorted **only if** the tree is a BST.

Preorder Traversal

Order: **node -> left -> right**

```
function preorder(root) {
  if (root == null) {
    return;
  }
  console.log(root.val);
  preorder(root.left);
  preorder(root.right);
}
```

Example order in diagram: `[4,3,2,6,5,7]`

Postorder Traversal

Order: **left -> right -> node**

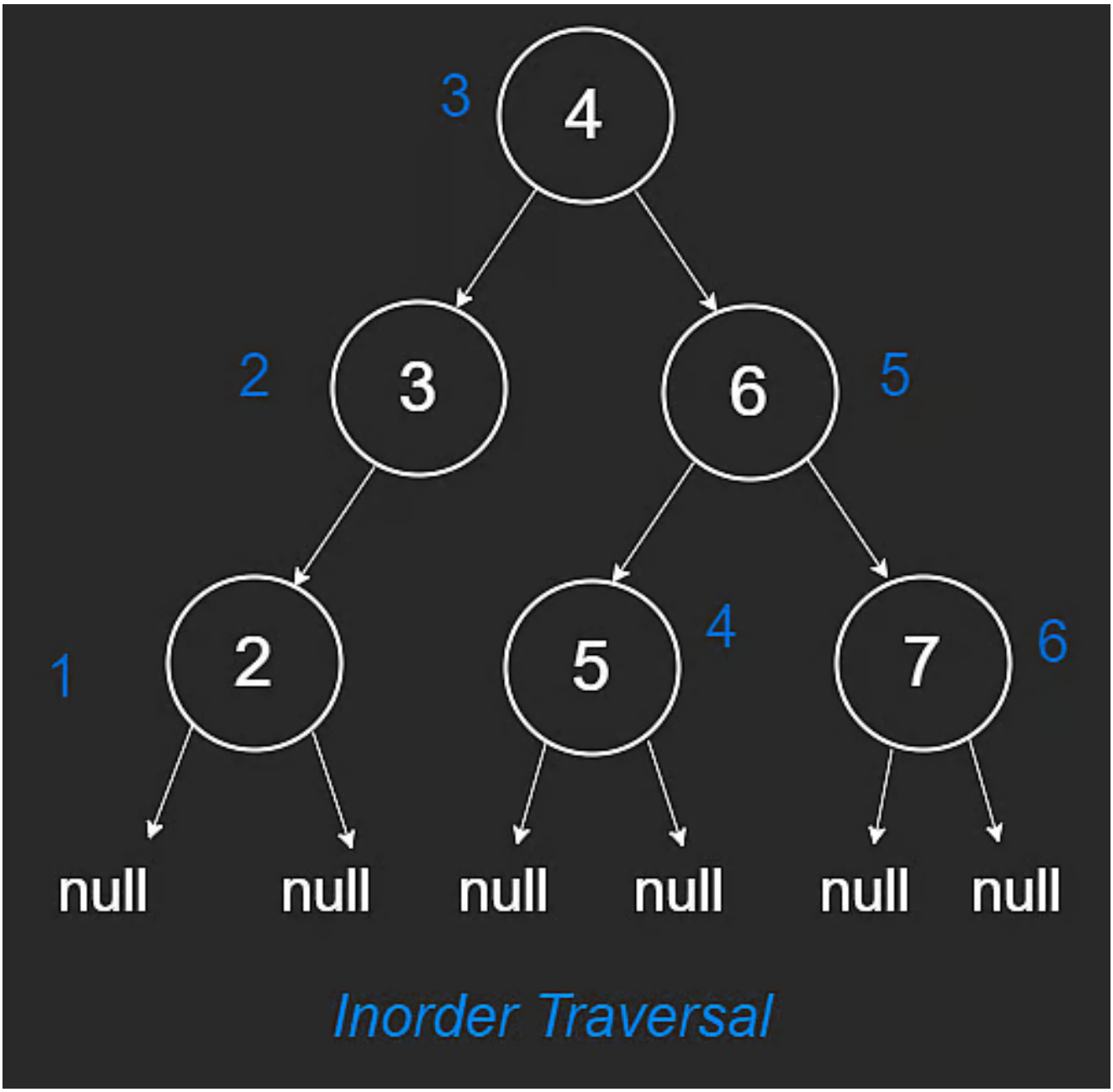


Figure 42: DFS inorder traversal

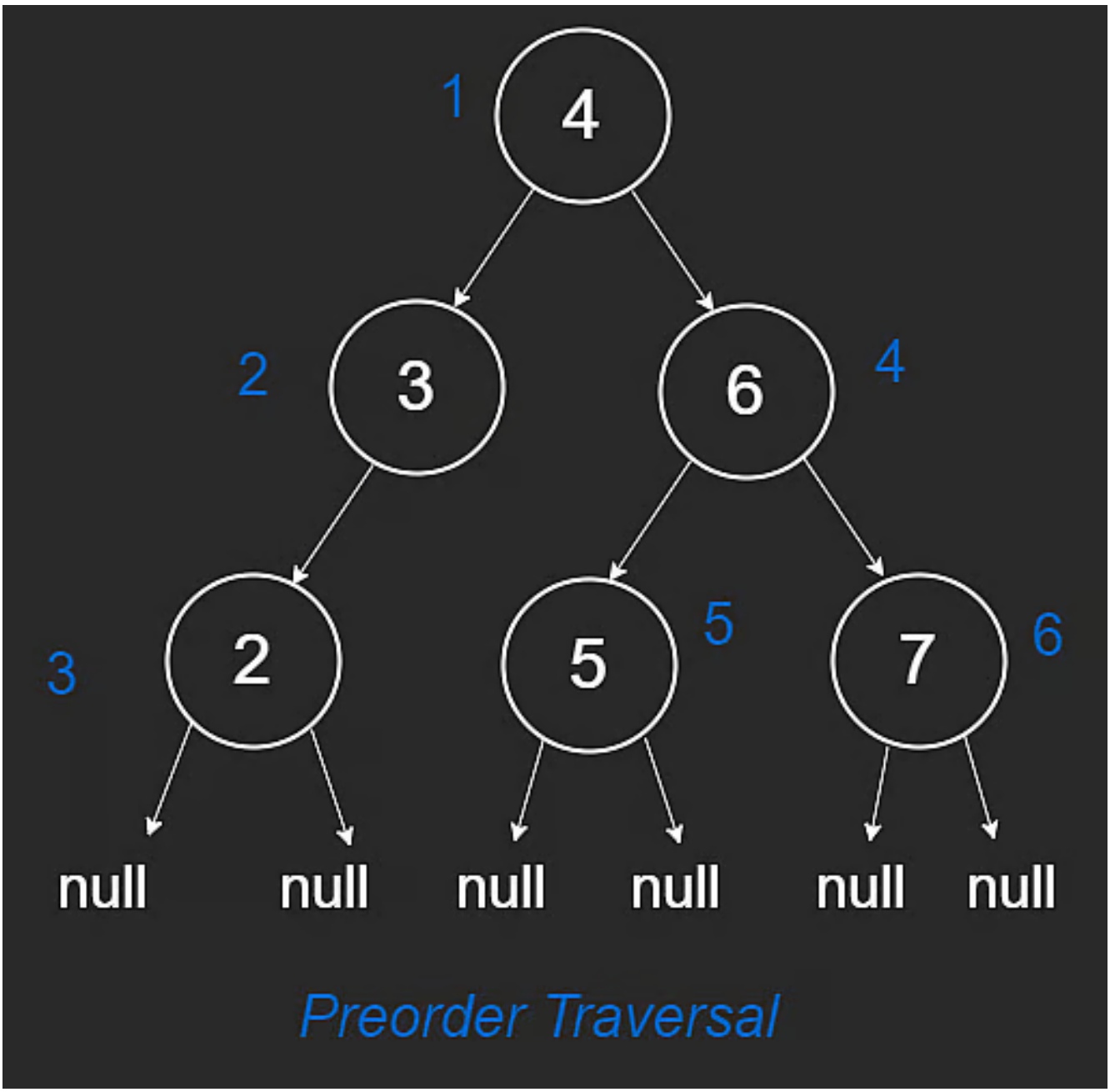


Figure 43: DFS preorder traversal

```
function postorder(root) {
  if (root == null) {
    return;
  }
  postorder(root.left);
  postorder(root.right);
  console.log(root.val);
}
```

Example order in diagram: `[2,3,5,7,6,4]`

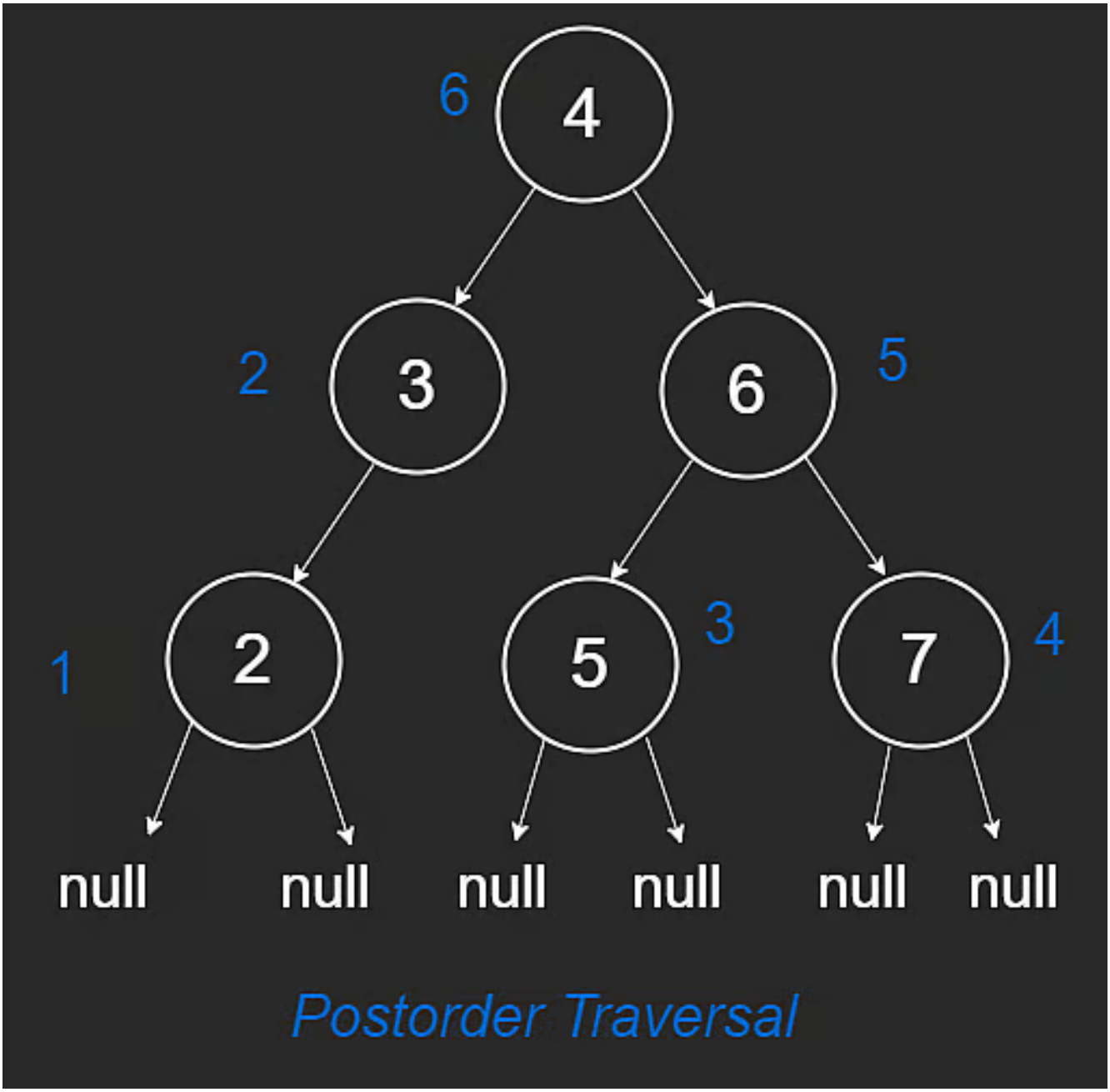


Figure 44: DFS postorder traversal

Time and Space Complexity

The following applies to all three traversals.

Time

Every node is visited once.

- Time: $O(n)$

where `n` is number of nodes.

Space

Recursive DFS uses call stack proportional to tree height `h`.

- Space: $O(h)$

So:

- balanced tree: $O(\log n)$
- skewed tree: $O(n)$

NeetCode Practice (Trees)

1) Kth Smallest Integer in BST

- Difficulty: **Medium**
- Pattern: **Inorder DFS with stack (iterative)**
- Core idea: Inorder traversal of a BST visits nodes in sorted order, so the k-th visited node is the answer.

Problem statement (summary): Given the root of a BST and integer `k` (1-indexed), return the k-th smallest value.

BST constraints:

- left subtree values are smaller than node value
- right subtree values are greater than node value
- both subtrees are BSTs

Examples:

- Input: `root = [2,1,3], k = 1`
- Output: `1`
- Input: `root = [4,3,5,2,null], k = 4`
- Output: `5`

Approach 1: Iterative DFS (Optimal)

Intuition: Use a stack to simulate recursive inorder traversal:

1. Push all left nodes.
2. Pop one node (next smallest).
3. Decrement `k`.
4. If `k == 0`, this node is the answer.
5. Move to right subtree and repeat.

This stops early at the k-th element instead of traversing all nodes.

Algorithm:

1. Initialize empty stack and `curr = root`.
2. While `stack` not empty or `curr` not null:
3. Push left chain.
4. Pop top node.
5. Decrement `k`; if zero, return node value.
6. Move `curr` to popped node's right child.

```
/**
 * Definition for a binary tree node.
 * class TreeNode {
 *     constructor(val = 0, left = null, right = null) {
 *         this.val = val;
 *         this.left = left;
 *         this.right = right;
 *     }
 * }
 */

class Solution {
  /**
   * @param {TreeNode} root
   * @param {number} k
   * @return {number}
   */
  kthSmallest(root, k) {
    const stack = [];
    let curr = root;

    while (stack.length > 0 || curr !== null) {
      while (curr !== null) {
        stack.push(curr);
        curr = curr.left;
      }
```

```
      curr = stack.pop();
      k--;

      if (k === 0) {
        return curr.val;
      }

      curr = curr.right;
    }
  }
}
```

Complexity:

- Time: $O(h + k)$ in the best stopping sense, up to $O(n)$ worst case
- Space: $O(h)$

Where:

- `n` is number of nodes
- `h` is tree height